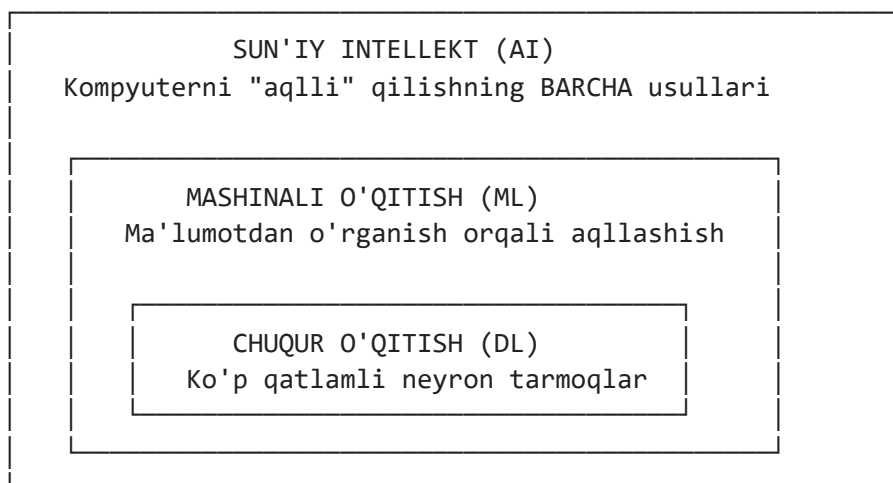

● SAVOL 16:

MASHINALI O'QITISH VA SUN'IY INTELLEKTNING FARQLARI. ML NING ASOSIY YO'NALISHLARI: SUPERVISED, UNSUPERVISED VA REINFORCEMENT LEARNING. HAYOTDAN KONKRET MISOLLAR.

1. Sun'iy Intellekt, ML va Deep Learning — Munosabati

Ko'pchilik bu uchta tushunchani aralashtirib yuboradi. Aslida ular bir-birining ichida joylashgan:



Muhim fakt: Har bir ML — bu AI, lekin har bir AI — bu ML emas. Masalan, shaxmat o'ynaydigan **minimax algoritmi** — bu AI, lekin ML emas, chunki u ma'lumotdan o'rganmaydi.

2. Sun'iy Intellekt (AI) nima?

Ta'rif: AI — kompyuterni insonlarga xos vazifalarni (fikrlash, muammo yechish, tilni tushunish) bajarishga qodir qiluvchi texnologiyalar majmui.

AI ning turlari (o'rganish usuliga qarab):

1. Rule-based AI → Qoidalar dasturchi tomonidan yoziladi
Misol: if-else, minimax, expert systems
Kamchiligi: yangi vaziyatga moslasha olmaydi
2. ML-based AI → Qoidalarni MA'LUMOTDAN o'zi o'rganadi
Misol: spam filtri, tavsiya tizimi
Afzalligi: yangi ma'lumotga moslashadi

Hayotiy farq:

Rule-based (eski usul):

```
if email.contains("lottery") or email.contains("win $"):  
    mark_as_spam()  
# Muammo: "Congratulations, you have WON!" – o'tib ketadi
```

```
ML-based (zamonaviy usul):  
model.predict(email) → spam / not_spam  
# 10 million spam emaildan o'rganib, 99.9% aniqlik bilan topadi
```

3. Mashinali O'qitish (ML) nima?

Ta'rif (Tom Mitchell, 1997): "Dastur T vazifasini P samaradorligi bilan bajarsa va E tajriba orqali P ning o'sishi kuzatilsa, u E tajribadan T vazifasida o'rganadi deyiladi."

Oddiyoq: ML — bu algoritmnining **misollardan qoida o'zi chiqarishi**.

Oddiy qiyoslash:

An'anaviy dasturlash:

Ma'lumot + Qoidalar → Natija

```
def uy_narxi(maydon, xonalar):  
    if maydon > 100 and xonalar >= 3:  
        return "qimmat"  
    else:  
        return "arzon"
```

Muammo: joylashuv, yil, holat hisobga olinmagan

ML dasturlash:

Ma'lumot + Natija → Qoidalarni o'zi topadi

```
from sklearn.linear_model import LinearRegression  
model = LinearRegression()  
model.fit(X_train, y_train)    # <- o'zining qoidasini topadi  
model.predict([[120, 3]])      # <- natija: 85000 dollar
```

4. AI va ML ning To'liq Taqqoslash Jadvali

Xususiyat	Sun'iy Intellekt (AI)	Mashinali O'qitish (ML)
Ta'rif	Aqli xulq ko'rsatuvchi tizim	Ma'lumotdan o'rganuvchi tizim
Maqsad	Insoniy vazifani bajarish	Bashorat / naqsh topish
O'rganish	Shart emas (qoidabazali bo'lishi mumkin)	Albatta ma'lumotdan o'rganadi
Qoidalar	Dasturchi yozadi	Model o'zi topadi
Yangi holatga	Moslasha olmaydi (rule-based)	Moslashadi
Misol	Shaxmat AI, GPS navigatsiya	Spam filtri, yuz tanish
Subset	Keng tushuncha	AI ning bir qismi

5. Supervised Learning — Nazoratli O'qitish

Ta'rif: Model **belgilangan (labeled)** ma'lumotdan o'rganadi. Ya'ni har bir kirishga mos to'g'ri javob (label) berilgan bo'ladi.

Mantiq:

Kirish (X) + To'g'ri javob (y) → Model o'rganadi → Yangi X uchun y bashorat qiladi

Analogiya:

O'qituvchi (X=misol, y=javob) → Talaba o'rganadi → Imtihon (yangi X) → Javob beradi

Supervised Learning ning 2 turi:

SUPERVISED LEARNING	
KLASSIFIKATSIYA (Classification)	REGRESSIYA (Regression)
Chiqish: kategoriya "spam / not spam"	Chiqish: son "narx: 85000 dollar"
"kasallik bor / yo'q"	"harorat: 23.5 daraja"
"it / mushuk / qush"	"sotuv: 1200 dona"

Hayotiy misollar:

KLASSIFIKATSIYA MISOLI: Spam aniqlash
Gmail, Outlook, Yandex.Mail da ishlatiladi

Ma'lumot qanday ko'rinadi:

<i># email_matn</i>	<i> spam?</i>
<i># "You won 1 million dollars!"</i>	<i> 1 (spam)</i>
<i># "Meeting tomorrow at 9am"</i>	<i> 0 (not spam)</i>
<i># "FREE! Click here to claim prize"</i>	<i> 1 (spam)</i>
<i># "Your invoice is attached"</i>	<i> 0 (not spam)</i>

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer
```

```
emaillar = [
    "You won 1 million dollars!",
    "Meeting tomorrow at 9am",
    "FREE! Click here to claim prize",
    "Your invoice is attached"
]
labellar = [1, 0, 1, 0] # 1=spam, 0=normal
```

```
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(emaillar)
```

```
model = MultinomialNB()
model.fit(X, labellar)
```

```
yangi = vectorizer.transform(["Win free money now!"])
print(model.predict(yangi)) # [1] → spam
```

REGRESSIYA MISOLI: Uy narxini bashorat qilish
Zillow, Realtor.com platformalarda ishlatiladi

```

# Ma'lumot qanday ko'rinadi:
# maydon(m2) | xonalar | toshkent_markaz(km) | narx(dollar)
#      85      | 3      | 5      | 75000
#     120      | 4      | 2      | 110000
#      60      | 2      | 10     | 45000

from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([
    [85, 3, 5],
    [120, 4, 2],
    [60, 2, 10],
    [95, 3, 3]
])
y = np.array([75000, 110000, 45000, 82000])

model = LinearRegression()
model.fit(X, y)

# Yangi uy narxini bashorat qilish
yangi_uy = np.array([[100, 3, 4]])
print(model.predict(yangi_uy)) # [88500.] (taxminan)

```

Supervised Learning algoritmlari:

Klassifikatsiya uchun:	Regressiya uchun:
Logistic Regression	Linear Regression
Decision Tree	Polynomial Regression
Random Forest	Ridge / Lasso
SVM	SVR
KNN	Gradient Boosting
Neural Networks	Neural Networks

6. Unsupervised Learning — Nazorat Qilinmagan O'qitish

Ta'rif: Model **belgisiz (unlabeled)** ma'lumotdan o'rganadi. To'g'ri javoblar berilmaydi — model ma'lumot ichidagi **yashirin naqshlarni** o'zi topadi.

Mantiq:

Faqat kirish (X) → Model naqsh / guruh topadi → Natija

Analogiya:

Kutubxonachi kitoblarni o'qimay, faqat qalinligi, rangi, sarlavha uzunligiga qarab guruhlariga ajratadi.
Kimdir aytmagan – o'zi mantiqqa ko'ra ajratgan.

Unsupervised Learning ning asosiy turlari:

UNSUPERVISED LEARNING		
KLASTERIZATSIYA	DIMENSIYA	ASSOTSIATSIYA

(Clustering)	KAMAYTIRISH (Dim. Reduction)	(Association)
O'xshashlarni guruhlash	Ko'p o'zgaruvchini kamaytirish	Birgalikda uchraydigan naqshlar
K-means	PCA	Apriori
DBSCAN	t-SNE	FP-Growth
Hierarchical	Autoencoder	

Hayotiy misollar:

```
# KLAUSTERIZATSIYA MISOLI: Mijozlarni segmentlash
# Amazon, Uzum bozor da marketing uchun ishlatiladi
```

```
# Ma'lumot qanday ko'rinadi:
# yosh | oylik_xarid(dollar) | tovar_turi
# 25 | 500 | elektronika
# 45 | 2000 | uy jihozlari
# 22 | 150 | kiyim
# 50 | 1800 | uy jihozlari
```

```
from sklearn.cluster import KMeans
import numpy as np
```

```
X = np.array([
    [25, 500],
    [45, 2000],
    [22, 150],
    [50, 1800],
    [24, 480],
    [47, 1950],
    [21, 130]
])
```

```
model = KMeans(n_clusters=3, random_state=42)
model.fit(X)
```

```
print(model.labels_)
# [1, 0, 2, 0, 1, 0, 2]
# Guruh 0: "premium mijozlar" (45-50 yosh, ko'p xarid)
# Guruh 1: "o'rta guruh" (25 yosh, o'rtacha xarid)
# Guruh 2: "iqtisodiy guruh" (21-22 yosh, kam xarid)
# → Marketing: har guruhga alohida taklif yuborish
# DIMENSIYA KAMAYTIRISH MISOLI: PCA
# Yuz tanish, signal qayta ishlash, ma'lumot vizualizatsiyasida
```

```
# Muammo: 100 ta o'zgaruvchi → grafik chizib bo'lmaydi
# Yechim: PCA → 2 ta o'zgaruvchiga kamaytirish
```

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

```
X = iris.data          # 4 ta o'zgaruvchi: sepal, petal uzunlik/kengligi

pca = PCA(n_components=2)
X_2d = pca.fit_transform(X) # 4D → 2D

print(f"Ma'lumot saqlash darajasi: {pca.explained_variance_ratio_.sum():.1%}")
# Ma'lumot saqlash darajasi: 97.8%
# → 4 o'zgaruvchidan 2 tasiga o'tib, 97.8% ma'lumot saqlanadi!
```

7. Reinforcement Learning — Mustahkamlovchi O'qitish

Ta'rif: Agent **muhit** bilan o'zaro ta'sir orqali o'rganadi. Har bir to'g'ri harakat uchun **mukofot (+)**, noto'g'ri harakat uchun **jazo (-)** oladi. Maqsad — umumiy mukofotni maksimallashtirishdir.

Tizim elementlari:

```
AGENT      → O'rganuvchi (robot, AI, dastur)
MUHIT      → Agent harakat qiladigan dunyo (o'yin, fabrika, yo'l)
HOLAT (s)  → Hozirgi vaziyat (koordinata, ball, pozitsiya)
HARAKAT(a) → Agent qiladigan ish (chapga bur, o'q ot, saqlash)
MUKOFOT(r) → Harakatga javob: +1 (yaxshi), -1 (yomon), 0 (neytral)
```

Tsikl:

```
Agent → Holat kuzatadi → Harakat qiladi →
Muhit → Yangi holat + Mukofot beradi → Agent o'rganadi
```

RL ning boshqa yo'nalishlardan farqi:

```
Supervised:    "To'g'ri javob berilgan" – o'qituvchi bor
Unsupervised:  "Javob yo'q" – naqsh topish
Reinforcement: "To'g'ri javob yo'q, faqat mukofot/jazo bor" – sinab
ko'rish orqali
```

Hayotiy misollar:

```
# 1. O'YIN O'YNASH – AlphaGo (Google DeepMind, 2016)
#   Go o'yinida dunyoning eng yaxshi o'yinchisini yengdi
#   Agent:   AlphaGo
#   Muhit:   Go taxtasi
#   Holat:   Taxtadagi toshlar joylashuvi
#   Harakat: Tosh qo'yish koordinatasi
#   Mukofot: +1 (g'alaba), -1 (yutqazish)

# 2. O'Z-O'ZI HAYDOVCHI MASHINA – Tesla Autopilot
#   Agent:   avtomobil boshqaruv tizimi
#   Muhit:   yo'l, boshqa mashinalar, pedestrianlar
#   Holat:   tezlik, yo'nalish, kameradan tasvir
#   Harakat: gaz, tormoz, rul burish
#   Mukofot: +1 (xavfsiz harakat), -1 (to'qnashuv, chiziqdan chiqish)

# 3. ROBOT O'QITISH – Boston Dynamics
#   Agent:   robot oyoqlari
#   Muhit:   notekis er, to'siqlar
```

```

# Holat: oyoqlar burchagi, balans, tezlik
# Harakat: har bir motor uchun kuch
# Mukofot: +1 (oldinga harakat), -1 (yiqilish)

# 4. SAVDO – Algorithmic Trading
# Agent: savdo algoritmi
# Muhit: birja bozori
# Holat: aksiya narxi, hajm, vaqt
# Harakat: sotib olish, sotish, kutish
# Mukofot: + (foyda), - (zarar)
# ODDIY RL MISOLI: OpenAI Gym bilan
import gym

env = gym.make('CartPole-v1') # Yog'och balanslashtirish o'yini
# Agent: aravachani boshqaradi
# Maqsad: yog'ichni tushirmay ushlab turish
# Harakat: chapga yoki o'ngga itarish (0 yoki 1)

observation = env.reset()
umumiy_mukofot = 0

for qadam in range(200):
    harakat = env.action_space.sample() # Random harakat (o'rganmagan)
    obs, mukofot, tugadi, info = env.step(harakat)
    umumiy_mukofot += mukofot

    if tugadi:
        break

print(f"Jami mukofot: {umumiy_mukofot}")
# Random: ~20 qadam ← o'rganmagan
# O'qitilgan agent: 200 qadam ← maqsadga erishgan

```

8. Uchala Yo'nalishning To'liq Taqqoslash Jadvali

Xususiyat	Supervised	Unsupervised	Reinforcement
Ma'lumot turi	Belgilangan (labeled)	Belgisiz (unlabeled)	Muhit bilan o'zaro ta'sir
To'g'ri javob	Ha, berilgan	Yo'q	Yo'q, mukofot/jazo bor
Maqsad	Bashorat qilish	Naqsh / guruh topish	Mukofotni maksimallashtirish
O'qituvchi	Bor (insonlar label qiladi)	Yo'q	Yo'q (muhit javob beradi)
Natija	Kategoriya yoki son	Guruhlar, tuzilmalar	Harakat siyosati (policy)
Qiyinlik	O'rta	O'rta	Eng qiyin
Ma'lumot miqdori	Ko'p labeled data	Ko'p unlabeled data	Ko'p sinov-xato iteratsiya
Algoritmlar	Linear Reg., SVM, RF	K-means, PCA, DBSCAN	Q-Learning, PPO, DQN
Real misol	Spam filtri, yuz tanish	Mijoz segmentatsiyasi	AlphaGo, Tesla Autopilot

9. Real Sanoatdagi Qo'llanish Misollar

Soha	Yo'nalish	Misol
Tibbiyot	Supervised	X-ray dan kasallik aniqlash
Tibbiyot	Unsupervised	Genlarga ko'ra guruhlanish
Moliya	Supervised	Kredit berish/bermaslik
Moliya	Reinforcement	Birja savdosi
Transport	Supervised	Yo'l belgisini aniqlash
Transport	Reinforcement	O'z-o'zi haydovchi mashina
Internet	Supervised	Spam filtri, tavsiya
Internet	Unsupervised	Foydalanuvchi guruhlash
O'yin	Reinforcement	AlphaGo, OpenAI Five
Robotika	Reinforcement	Boston Dynamics Walk
Harbiy	Supervised	Drone orqali obyekt aniqlash
Harbiy	Reinforcement	Taktik qaror qabul qilish

10. Python da Uchala Yo'nalish — Qisqa Kod

```

from sklearn.linear_model import LogisticRegression # Supervised
from sklearn.cluster import KMeans                # Unsupervised
# Reinforcement uchun: pip install stable-baselines3

import numpy as np

X = np.array([[1,2],[2,3],[8,9],[9,10],[3,2],[8,8]])
y = np.array([0, 0, 1, 1, 0, 1]) # Label bor → Supervised

# — 1. Supervised —————
sup_model = LogisticRegression()
sup_model.fit(X, y)
print("Supervised bashorat:", sup_model.predict([[5, 5]]))
# Supervised bashorat: [1] ← model labeldan o'rgandi

# — 2. Unsupervised —————
unsup_model = KMeans(n_clusters=2, random_state=42, n_init=10)
unsup_model.fit(X) # y ishlatilmaydi!
print("Unsupervised guruhlar:", unsup_model.labels_)
# Unsupervised guruhlar: [0 0 1 1 0 1] ← o'zi topdi

# — 3. Reinforcement (konseptual) —————
# from stable_baselines3 import PPO
# import gym
# env = gym.make("CartPole-v1")
# model = PPO("MLpPolicy", env, verbose=0)
# model.learn(total_timesteps=10000) # 10000 qadam sinov-xato

```

11. Xulosa

SUN'IY INTELLEKT (AI)

- Kompyuterga aqlli xulq beruvchi BARCHA texnikalar
- Rule-based AI: qoidalar dasturchi tomonidan yoziladi
- ML-based AI: qoidalar ma'lumotdan o'rganiladi

MASHINALI O'QITISH (ML) – AI ning bir qismi

└─ Ma'lumotdan o'rganib bashorat qiluvchi algoritmlar

— SUPERVISED LEARNING

Labeled data → O'rganish → Bashorat

Klassifikatsiya: kategoriya chiqaradi (spam / not spam)

Regressiya: son chiqaradi (85000 dollar)

Algoritm: LogReg, Decision Tree, Random Forest, SVM

— UNSUPERVISED LEARNING

Unlabeled data → Naqsh / guruh topish

Klasterizatsiya: guruhga ajratadi (3 ta mijoz turi)

Dimensiya kamaytirish: soddalashtiadi (100D → 2D)

Algoritm: K-means, DBSCAN, PCA, t-SNE

— REINFORCEMENT LEARNING

Muhit + Mukofot/Jazo → Harakat siyosatini o'rganish

Agent → Harakat qiladi → Mukofot oladi → Yaxshilanadi

Algoritm: Q-Learning, DQN, PPO, A3C

💡 **Eslab qol: Supervised** — o'qituvchi bor, javoblar berilgan. **Unsupervised** — o'qituvchi yo'q, naqsh o'zi topiladi. **Reinforcement** — o'qituvchi yo'q, lekin to'g'ri harakat uchun mukofot beriladi. Supervised eng ko'p ishlatiladi (real loyihalarning ~70%), Reinforcement eng qiyin va hisoblash resursi talab qiladi.

● SAVOL 17:

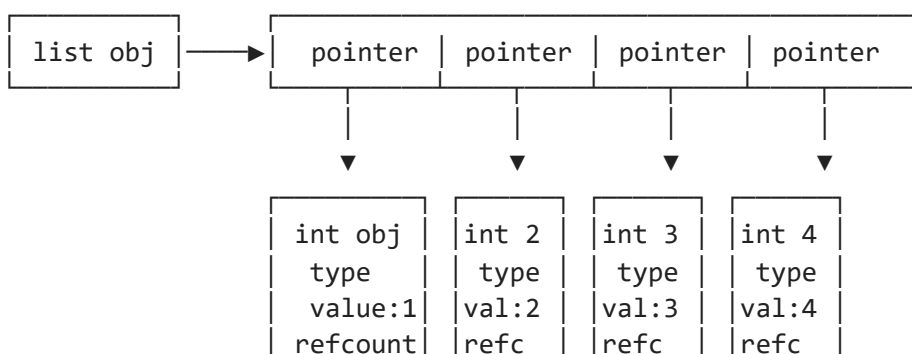
NUMPY MASSIVLARI VA PYTHON RO'YXATLARINI SOLISHTIRISH: XOTIRA SAMARADORLIGI, VEKTORLASHTIRILGAN AMALLAR, BROADCASTING MEXANIZMI VA ML DA NIMA UCHUN NUMPY AFZAL EKANLIGI.

1. Xotira Tuzilmasi — Asosiy Farq

Python ro'yxati va NumPy massivi xotirada **tamomila boshqacha** saqlanadi. Bu farqni tushunish hamma narsaning asosi.

PYTHON RO'YXATI xotirada:

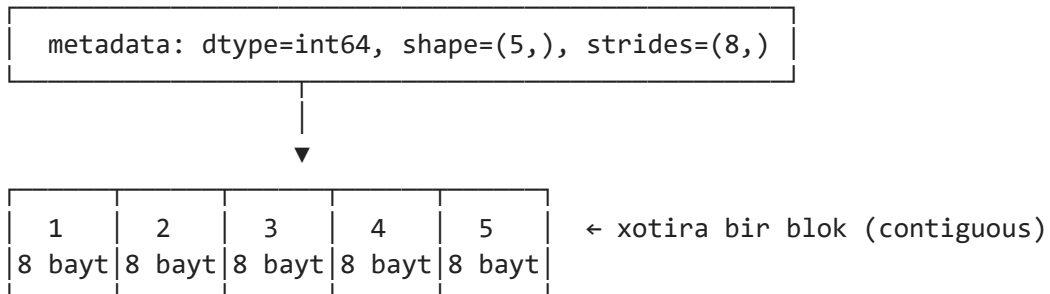
```
list = [1, 2, 3, 4, 5]
```



Har bir int ob'yekti: 28 bayt (type + value + refcount)
5 ta element: $5 \times 28 + 56$ (list overhead) = 196 bayt

NUMPY MASSIVI xotirada:

```
arr = np.array([1, 2, 3, 4, 5])
```



5 ta element: $5 \times 8 = 40$ bayt + 96 (metadata) = 136 bayt

Fakt: Python ro'yxatida har element — bu alohida ob'yekt (pointer + type + refcount). NumPy da esa barcha elementlar **bir xil turdagi** va **ketma-ket joylashgan** — bu kesh samaradorligini keskin oshiradi.

Xotira o'lchamini o'lchash:

```
import numpy as np
import sys

# Python ro'yxati
python_list = list(range(1_000_000))

# NumPy massivi
numpy_array = np.arange(1_000_000, dtype=np.int64)

# Xotira hajmi
list_memory = sys.getsizeof(python_list) + \
               sum(sys.getsizeof(x) for x in python_list)
numpy_memory = numpy_array.nbytes

print(f"Python ro'yxati : {list_memory / 1024 / 1024:.1f} MB")
print(f"NumPy massivi   : {numpy_memory / 1024 / 1024:.1f} MB")
print(f"Nisbat          : {list_memory / numpy_memory:.1f}x kam xotira")

# Natija:
# Python ro'yxati : 28.9 MB
# NumPy massivi   : 7.6 MB
# Nisbat          : 3.8x kam xotira
```

NumPy dtype — xotira nazorati:

```
import numpy as np

# Bir xil ma'lumot, turli xotira sarfi:
a_int8 = np.array([1, 2, 3], dtype=np.int8) # 1 bayt/element [-128, 127]
```

```

a_int16 = np.array([1, 2, 3], dtype=np.int16) # 2 bayt/element [-32768, 32767]
a_int32 = np.array([1, 2, 3], dtype=np.int32) # 4 bayt/element [-2^31, 2^31-1]
a_int64 = np.array([1, 2, 3], dtype=np.int64) # 8 bayt/element [-2^63, 2^63-1]
a_float32 = np.array([1, 2, 3], dtype=np.float32) # 4 bayt/element ~7 xona aniqlik
a_float64 = np.array([1, 2, 3], dtype=np.float64) # 8 bayt/element ~15 xona aniqlik

print(a_int8.nbytes) # 3 bayt
print(a_float64.nbytes) # 24 bayt

```

ML da muhim:

Neyron tarmoqlarda float32 ishlatiladi → float64 ga nisbatan 2x kam xotira

Bu GPU da katta modellarga sig'ishga imkon beradi

2. Tezlik Farqi — Vektorlashtirilgan Amallar

Nima uchun NumPy tez?

Python ro'yxatida arifmetika:

```

for i in range(len(list)):
    result[i] = list[i] * 2

```

Har iteratsiyada:

1. Python interpretator for-loop ni boshqaradi (overhead)
2. `list[i]` → pointer → ob'yekt → qiymatni olish
3. `* 2` → yangi int ob'yekt yaratish
4. `result[i] =` yangi ob'yektga pointer saqlash
→ GIL (Global Interpreter Lock) ham to'sqinlik qiladi

NumPy da vektorlashtirilgan amal:

```
result = arr * 2
```

1. NumPy C da yozilgan loop ishga tushadi (Python emas!)
2. Xotira bir blok → CPU keshi to'liq foydalanadi
3. SIMD (Single Instruction Multiple Data) buyruqlari:
bir buyruq bilan 4-8 ta element parallel hisoblanadi
→ Python GIL chetlab o'tiladi

Tezlikni o'lchash:

```

import numpy as np
import time

```

```
n = 10_000_000 # 10 million element
```

```
python_list = list(range(n))
numpy_array = np.arange(n, dtype=np.float64)
```

Python ro'yxati bilan

```

boshlanish = time.time()
natija = [x * 2 + 1 for x in python_list]
python_vaqt = time.time() - boshlanish

```

```
# NumPy bilan
boshlanish = time.time()
natija = numpy_array * 2 + 1
numpy_vaqt = time.time() - boshlanish

print(f"Python ro'yxati: {python_vaqt:.3f} sek")
print(f"NumPy massivi : {numpy_vaqt:.4f} sek")
print(f"NumPy {python_vaqt/numpy_vaqt:.0f}x tezroq")

# Natija:
# Python ro'yxati: 1.842 sek
# NumPy massivi : 0.023 sek
# NumPy 80x tezroq
```

Vektorlashtirilgan amallar — to'g'ri va noto'g'ri usul:

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
b = np.array([ 1,  2,  3,  4,  5])

# =====
# ❌ NOTO'G'RI: Python Loop (sekin)
# =====
natija = []
for i in range(len(a)):
    natija.append(a[i] + b[i])
print(natija)    # [11, 22, 33, 44, 55]

# =====
# ✅ TO'G'RI: Vektorlashtirilgan (tez)
# =====
natija = a + b
print(natija)    # [11 22 33 44 55] ← bir qator, 80x tezroq

# Barcha amallar vektorizatsiya bo'ladi:
print(a + b)      # [11 22 33 44 55]   qo'shish
print(a - b)      # [ 9 18 27 36 45]   ayirish
print(a * b)      # [10 40 90 160 250] ko'paytirish
print(a / b)      # [10. 10. 10. 10. 10.] bo'lish
print(a ** 2)     # [100 400 900 1600 2500] daraja
print(np.sqrt(a)) # [3.16 4.47 5.47 6.32 7.07] ildiz
print(np.exp(b))  # [2.71 7.38 20.08 54.6 148.4] e^x
print(np.log(a))  # [2.30 2.99 3.40 3.68 3.91] ln(x)
print(a > 25)     # [False False True True True] shart
print(a[a > 25])  # [30 40 50]   shartli indekslash
```

Universal funksiyalar (ufunc):

```
import numpy as np

arr = np.array([-3, -1, 0, 2, 5, -4, 8])

# NumPy universal funksiyalari — C darajasida tez:
print(np.abs(arr))    # [3 1 0 2 5 4 8]   mutlaq qiymat
```

```
print(np.sin(arr))           # sin funksiyasi
print(np.maximum(arr, 0))    # [0 0 0 2 5 0 8]  ReLU! (ML da keng ishlatiladi)
print(np.sum(arr))           # 7      yig'indi
print(np.mean(arr))          # 1.0    o'rtacha
print(np.std(arr))           # 3.77  standart og'ish
print(np.min(arr))           # -4    minimum
print(np.argmax(arr))        # 6     maksimum indeks
print(np.cumsum(arr))        # [-3 -4 -4 -2  3 -1  7] kumulativ yig'indi
```

3. Broadcasting Mexanizmi

Ta'rif: Broadcasting — har xil o'lchamdagi massivlar orasida **nusxa ko'chirmasdan** arifmetik amallar bajarish mexanizmi. NumPy kichik massivni katta massiv o'lchamiga **mantiqan** kengaytiradi.

Broadcasting qoidalari (3 ta):

Qoida 1: Agar massivlarning o'lchami (ndim) farqli bo'lsa, kichik massiv chap tomondan 1 lar bilan to'ldiriladi.
 $(2,3)$ va $(3,)$ $\rightarrow (2,3)$ va $(1,3) \rightarrow (2,3)$ va $(2,3)$

Qoida 2: O'lcham bo'yicha 1 bo'lgan o'q katta massivning o'sha o'qi bo'yicha ko'paytiriladi (nusxa ko'chiriladi).
 $(1,3) \rightarrow (4,3) \leftarrow 4$ marta ko'paytiriladi (xotirada emas!)

Qoida 3: O'lchamlar mos kelmasa va hech biri 1 bo'lmasa $\rightarrow$ XATO.
 $(2,3)$ va $(2,4) \rightarrow$ ValueError: shapes not aligned

Oddiy broadcasting:

```
import numpy as np
```

```
# — Misol 1: Skalyar bilan —————
arr = np.array([1, 2, 3, 4, 5])
print(arr + 10)    # [11 12 13 14 15]
# Aslida: [1,2,3,4,5] + [10,10,10,10,10]
# Lekin [10,10,10,10,10] xotirada YARATILMADI — broadcasting!
```

```
# — Misol 2: Vektor bilan —————
matritsa = np.array([          # shape: (3, 4)
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9,10,11,12]
])
vektor = np.array([10, 20, 30, 40]) # shape: (4,)  $\rightarrow$  (1,4)  $\rightarrow$  (3,4)

print(matritsa + vektor)
# [[11 22 33 44]
#  [15 26 37 48]
#  [19 30 41 52]]
# vektor har qatorga qo'shildi — Lekin nusxa ko'chirilmadi!
```

```
# — Misol 3: Ustun vektor bilan —————
ustun = np.array([[10],      # shape: (3,1)  $\rightarrow$  (3,4)
    [20],
```

```

[30]])

print(matritsa + ustun)
# [[11 12 13 14]
#  [25 26 27 28]
#  [39 40 41 42]]
# ustun har ustunga qo'shildi!

```

Broadcasting vizualizatsiyasi:

MISOL: (3,4) matritsa + (4,) vektor

Matritsa (3,4):		Vektor (4,):		Natija (3,4):																								
<table><tr><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>5</td><td>6</td><td>7</td><td>8</td></tr><tr><td>9</td><td>10</td><td>11</td><td>12</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	+	→→→	=	<table><tr><td>11</td><td>22</td><td>33</td><td>44</td></tr><tr><td>15</td><td>26</td><td>37</td><td>48</td></tr><tr><td>19</td><td>30</td><td>41</td><td>52</td></tr></table>	11	22	33	44	15	26	37	48	19	30	41	52
1	2	3	4																									
5	6	7	8																									
9	10	11	12																									
11	22	33	44																									
15	26	37	48																									
19	30	41	52																									
		<table><tr><td>10</td><td>20</td><td>30</td><td>40</td></tr><tr><td>10</td><td>20</td><td>30</td><td>40</td></tr><tr><td>10</td><td>20</td><td>30</td><td>40</td></tr></table>	10	20	30	40	10	20	30	40	10	20	30	40														
10	20	30	40																									
10	20	30	40																									
10	20	30	40																									

vektor 3 marta ko'paytirilgan kabi ishladi (xotirada emas!)

ML da Broadcasting — real ishlatilishi:

```

import numpy as np

# — Normalizatsiya (ML preprocessing) —————
# 1000 ta namunaviy ma'lumot, 4 ta xususiyat
np.random.seed(42)
X = np.random.randn(1000, 4) * np.array([10, 100, 1, 50])

mean = X.mean(axis=0) # shape: (4,) — har ustun o'rtachasi
std = X.std(axis=0) # shape: (4,) — har ustun og'ishi

# Broadcasting ishlatiladi: (1000,4) - (4,) → (1000,4)
X_norm = (X - mean) / std

print(f"X o'rtachasi : {X.mean(axis=0).round(1)}")
print(f"Norm o'rtachasi: {X_norm.mean(axis=0).round(10)}")
print(f"Norm std : {X_norm.std(axis=0).round(2)}")

# Natija:
# X o'rtachasi : [ 0.5  0.3 -0.1  0.8]
# Norm o'rtachasi: [0. 0. 0. 0.]
# Norm std : [1. 1. 1. 1.]

# — Neyron tarmoq oldingi o'tish (Forward pass) —————
# batch_size=32, input_features=4, neurons=8

W = np.random.randn(4, 8) # og'irliklar, shape: (4, 8)
b = np.random.randn(8) # bias, shape: (8,)
X_batch = np.random.randn(32, 4) # kirishlar, shape: (32, 4)

# (32,4) @ (4,8) → (32,8) matritsa ko'paytmasi
Z = X_batch @ W # shape: (32, 8)

```

```
# (32,8) + (8,) → broadcasting → (32,8)
Z = Z + b # har qatorga bias qo'shildi!
A = np.maximum(Z, 0) # ReLU aktivatsiya
print(f"Chiqish shakli: {A.shape}") # (32, 8)
```

4. Ko'p O'lchamli Massivlar (ndarray)

```
import numpy as np
```

```
# — 1D: vektor —————
v = np.array([1, 2, 3, 4, 5])
print(v.shape) # (5,)
print(v.ndim) # 1
```

```
# — 2D: matritsa —————
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.shape) # (2, 3)
print(m.ndim) # 2
```

```
# Amallar:
print(m.T) # Transpose: (3,2)
print(m.sum()) # 21 — barcha elementlar
print(m.sum(axis=0)) # [5 7 9] — ustunlar bo'yicha
print(m.sum(axis=1)) # [6 15] — qatorlar bo'yicha
print(m.reshape(3, 2)) # (3,2) ga o'zgartirish
print(m.flatten()) # [1 2 3 4 5 6] — 1D ga
```

```
# — 3D: tasvirlar (ML da) —————
# Rasm: (balandlik, kengligi, kanallar)
rasm = np.random.randint(0, 255, size=(64, 64, 3), dtype=np.uint8)
print(rasm.shape) # (64, 64, 3)
```

```
# Mini-batch: (namunalar, balandlik, kengligi, kanallar)
batch = np.random.randint(0, 255, size=(32, 64, 64, 3), dtype=np.uint8)
print(batch.shape) # (32, 64, 64, 3)
print(f"Xotira: {batch.nbytes / 1024 / 1024:.1f} MB") # 12.0 MB
```

```
# — Matritsa ko'paytmasi —————
A = np.array([[1, 2],
              [3, 4]]) # (2,2)
B = np.array([[5, 6],
              [7, 8]]) # (2,2)

print(A @ B) # Matritsa ko'paytmasi
# [[19 22]
#  [43 50]]

print(A * B) # Element-wise ko'paytma (broadcasting)
# [[ 5 12]
#  [21 32]]
```

5. Indeksash va Kesish (Slicing)

```

import numpy as np

arr = np.arange(20).reshape(4, 5)
print(arr)
# [[ 0  1  2  3  4]
#   [ 5  6  7  8  9]
#  [10 11 12 13 14]
#  [15 16 17 18 19]]

# — Oddiy indekslash —————
print(arr[1, 3])      # 8          - 2-qator, 4-ustun
print(arr[0])         # [0 1 2 3 4] - 1-qator
print(arr[:, 2])      # [2 7 12 17] - 3-ustun

# — Slice —————
print(arr[1:3, 1:4])
# [[ 6  7  8]
#   [11 12 13]]

# — Boolean indekslash (ML da filtrlash uchun) —————
print(arr[arr > 10])   # [11 12 13 14 15 16 17 18 19]
print(arr[(arr > 5) & (arr < 15)]) # [ 6  7  8  9 10 11 12 13 14]

# — Fancy indekslash —————
indekslar = np.array([0, 2, 3])
print(arr[indekslar])
# [[ 0  1  2  3  4]
#   [10 11 12 13 14]
#   [15 16 17 18 19]]

# — View vs Copy — MUHIM FARQ —————
a = np.arange(10)
b = a[2:5]          # ← bu VIEW (ko'rinish), nusxa EMAS!
b[0] = 99
print(a)            # [ 0  1 99  3  4  5  6  7  8  9]
                    # ← a ham o'zgardi! b — a ning bir qismi

c = a[2:5].copy()   # ← bu COPY (nusxa)
c[0] = 0
print(a)            # o'zgarmadi ← c — mustaqil nusxa

```

6. ML da NumPy — Amaliy Qo'llanish

```

import numpy as np

# — 1. Train/Test split (sklearn siz) —————
np.random.seed(42)
X = np.random.randn(1000, 5)
y = np.random.randint(0, 2, 1000)

indekslar = np.arange(1000)
np.random.shuffle(indekslar)

ajratish = int(0.8 * 1000)          # 80%/20%
X_train, X_test = X[indekslar[:ajratish]], X[indekslar[ajratish:]]

```

```

y_train, y_test = y[indekslar[:ajratish]], y[indekslar[ajratish:]]

print(f"Train: {X_train.shape}, Test: {X_test.shape}")
# Train: (800, 5), Test: (200, 5)

# — 2. Normalizatsiya (Z-score) —————
mean = X_train.mean(axis=0) # faqat train dan hisoblash!
std = X_train.std(axis=0)
X_train_norm = (X_train - mean) / std
X_test_norm = (X_test - mean) / std # train parametrlari bilan!

# — 3. Linear Regression (NumPy siz sklearn) —————
# Normal tenglamasi:  $W = (X^T X)^{-1} X^T y$ 
np.random.seed(0)
X_lr = np.column_stack([np.ones(100), np.random.randn(100)]) # bias + feature
y_lr = 3 * X_lr[:, 1] + 2 + np.random.randn(100) * 0.5

W = np.linalg.inv(X_lr.T @ X_lr) @ X_lr.T @ y_lr
print(f"Intercept: {W[0]:.3f}") # ~2.000
print(f"Slope : {W[1]:.3f}") # ~3.000

# — 4. Confusion Matrix —————
def confusion_matrix_np(y_haqiqiy, y_bashorat, n_klass=2):
    cm = np.zeros((n_klass, n_klass), dtype=int)
    for h, b in zip(y_haqiqiy, y_bashorat):
        cm[h, b] += 1
    return cm

y_true = np.array([0, 1, 1, 0, 1, 0, 1, 1])
y_pred = np.array([0, 1, 0, 0, 1, 1, 1, 1])
cm = confusion_matrix_np(y_true, y_pred)
print(cm)
# [[2 1] ← [TN FP]
# [1 4] ← [FN TP]

# — 5. One-Hot Encoding —————
labellar = np.array([0, 2, 1, 2, 0]) # 3 klass
one_hot = np.eye(3)[labellar]
print(one_hot)
# [[1. 0. 0.]
# [0. 0. 1.]
# [0. 1. 0.]
# [0. 0. 1.]
# [1. 0. 0.]]

```

7. Python Ro'yxati vs NumPy — To'liq Taqqoslash

Xususiyat	Python <code>list</code>	NumPy <code>ndarray</code>
Xotira tuzilmasi	Har element — alohida ob'yekt	Bir blok, ketma-ket
Element turi	Aralash (<code>[1, "a", True]</code>)	Bir xil (<code>dtype</code>)
Xotira sarfi	Ko'p (pointer + ob'yekt)	Kam (faqat qiymat)
Arifmetika	Loop kerak	Vektorlashtirilgan

Xususiyat	Python <code>list</code>	NumPy <code>ndarray</code>
Tezlik	Sekin (Python interpreter)	Tez (C/Fortran)
Broadcasting	Yo'q	Bor
Ko'p o'lcham	Ichma-ich list	ndarray (shape)
Matematik	<code>import math</code>	O'zi (ufunc)
Xotira almashish	<code>copy()</code> kerak	View / Copy tanlash
GPU support	Yo'q	CuPy (NumPy API)
Sklearn input	Qabul qilmaydi	Qabul qiladi
Indekslash	<code>a[i]</code> faqat	Boolean, fancy, slice

8. NumPy ML da Nima Uchun Afzal — 5 ta Asosiy Sabab

SABAB 1: TEZLIK

10 million elementda:
 Python loop : 1.842 sek
 NumPy : 0.023 sek
 → 80x tezroq – C va SIMD buyruqlari tufayli

SABAB 2: XOTIRA

1 million int element:
 Python list : 28.9 MB
 NumPy int64 : 7.6 MB (3.8x kam)
 NumPy int8 : 1.0 MB (28.9x kam)
 → dtype tanlash bilan nazorat qilish mumkin

SABAB 3: BROADCASTING

(1000,4) - (4,) → nusxa ko'chirmasdan ishlaydi
 ML preprocessing, batch normalizatsiya uchun zarur

SABAB 4: EKOSISTEMA

Sklearn → NumPy array talab qiladi
 Pandas → NumPy ustida qurilgan
 TensorFlow → `tf.Tensor` ↔ `np.array`
 PyTorch → `torch.Tensor` ↔ `np.array` (`to_numpy()`)
 Matplotlib → NumPy array ni to'g'ridan to'g'ri qabul qiladi

SABAB 5: LINEER ALGEBRA

`np.linalg.inv()` → teskari matritsa
`np.linalg.eig()` → eigen qiymatlar
`np.linalg.svd()` → SVD (PCA uchun kerak)
`A @ B` → matritsa ko'paytmasi (neyron tarmoq asosi)

9. Xulosa

PYTHON RO'YXATI vs NUMPY MASSIVI:

list:

- Har element: pointer (8B) + ob'yekt (28B) = 36 bayt
- Aralash turdagi elementlar saqlash mumkin
- Arifmetika uchun Python loop kerak → sekin
- Broadcasting yo'q
- ML kutubxonalari qabul qilmaydi

ndarray:

- Har element: faqat qiymat (int8=1B, float32=4B, float64=8B)
- Faqat bir xil turdagi elementlar (dtype)
- Arifmetika: C darajasida, SIMD bilan → 80x tezroq
- Broadcasting: har xil o'lchamlar bilan ishlash
- Sklearn, Pandas, TensorFlow, PyTorch asosi

BROADCASTING qoidalari:

1. ndim farqli → kichikni chap tomondan 1 bilan to'ldirish
2. O'lcham 1 bo'lgan o'q → katta o'lchamga kengaytirish
3. Mos kelmaydigan o'lcham (va hech biri 1 emas) → ValueError

ML DA QANDAY ISHLATILADI:

- Normalizatsiya: $X_{\text{norm}} = (X - \text{mean}) / \text{std}$
- One-hot encoding: `np.eye(n_klass)[labellar]`
- Train/test split: `X[indekslar[:n]]`
- Forward pass: $Z = X @ W + b$
- Aktivatsiya: `np.maximum(Z, 0)` ← ReLU
- Matritsa: `np.linalg.inv()`, `.T`, `@`

💡 **Eslab qol:** NumPy da `b = a[2:5]` yozganingda **nusxa emas, view** hosil bo'ladi — `b` ni o'zgartirsang `a` ham o'zgaradi. Nusxa kerak bo'lsa: `b = a[2:5].copy()`. Bu ML da katta datasetlar bilan ishlashda xotira tejashning asosiy usuli — lekin ehtiyotkorlik zarur!

● SAVOL 18:

PANDAS DATAFRAME ASOSIY OPERATSIYALARI: MA'LUMOTLARNI YUKLASH, TOZALASH (NAN QIYMATLAR), FILTRLASH, GURUHLASH (GROUPBY) VA AGREGATSIYA. REAL DATASETDA AMALIY QO'LLASH.

1. Pandas va DataFrame — Asosiy Tushuncha

Pandas — Python da jadval ko'rinishidagi ma'lumotlar bilan ishlash uchun eng keng tarqalgan kutubxona. NumPy ustida qurilgan, lekin undan farqli ravishda **nomlangan ustunlar, aralash turdagi ma'lumotlar** va **indekslar** bilan ishlaydi.

Pandas ning asosiy tuzilmalari:

- Series → 1 o'lchamli, nomlangan massiv (bitta ustun)
- DataFrame → 2 o'lchamli, nomlangan jadval (bir nechta ustun)

DataFrame tuzilmasi:

index	ism	yosh	shahar	maosh
0	Ali	25	Toshkent	5000
1	Vali	32	Samarqand	7200
2	Hasan	28	Toshkent	6100
3	Zulfiya	35	Buxoro	8500

|
└─ Index

|
└─ str

|
└─ str

|
└─ int

(avtomatik yoki qo'lda)

Munosabat NumPy bilan:

DataFrame ichida har ustun → NumPy array (dtype bilan)

DataFrame.values → NumPy 2D array

DataFrame.to_numpy() → sklearn uchun NumPy array

2. DataFrame Yaratish va Ma'lumot Yuklash

To'g'ridan-to'g'ri yaratish:

```
import pandas as pd
```

```
import numpy as np
```

— Usul 1: Dictionary dan

```
data = {
    'ism'      : ['Ali', 'Vali', 'Hasan', 'Zulfiya', 'Malika'],
    'yosh'     : [25, 32, 28, 35, 29],
    'shahar'   : ['Toshkent', 'Samarqand', 'Toshkent', 'Buxoro', 'Toshkent'],
    'maosh'    : [5000, 7200, 6100, 8500, 5800]
}
df = pd.DataFrame(data)
print(df)
```

```
#      ism  yosh  shahar  maosh
# 0  Ali    25  Toshkent  5000
# 1  Vali    32  Samarqand  7200
# 2  Hasan   28  Toshkent  6100
# 3  Zulfiya  35    Buxoro  8500
# 4  Malika   29  Toshkent  5800
```

— Usul 2: NumPy array dan

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
df2 = pd.DataFrame(arr, columns=['A', 'B', 'C'])
```

— Usul 3: Ro'yxatlar ro'yxatidan

```
df3 = pd.DataFrame(
    [['Ali', 25], ['Vali', 32], ['Hasan', 28]],
    columns=['ism', 'yosh']
)
```

Fayldan yuklash:

```
import pandas as pd
```

```

# — CSV fayl —————
df = pd.read_csv('data.csv')
df = pd.read_csv('data.csv', sep=';') # ajratuvchi nuqta-vergul
df = pd.read_csv('data.csv', encoding='utf-8') # kodlash
df = pd.read_csv('data.csv', index_col='id') # indeks ustun
df = pd.read_csv('data.csv', nrows=1000) # faqat 1000 qator
df = pd.read_csv('data.csv', usecols=['ism', 'maosh']) # faqat bu ustunlar
df = pd.read_csv('data.csv', na_values=['N/A', '-', '?']) # NaN sifatida

# — Excel fayl —————
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
df = pd.read_excel('data.xlsx', sheet_name=0) # birinchi varaq

# — JSON fayl —————
df = pd.read_json('data.json')

# — SQL —————
import sqlite3
conn = sqlite3.connect('database.db')
df = pd.read_sql('SELECT * FROM employees', conn)
conn.close()

# — URL dan (to'g'ridan-to'g'ri internet) —————
url = 'https://raw.githubusercontent.com/datasets/iris/main/data/iris.csv'
df = pd.read_csv(url)

```

Faylga saqlash:

```

# CSV ga saqlash
df.to_csv('natija.csv', index=False) # index=False → indeks ustun
sifatida yozilmaydi
df.to_csv('natija.csv', encoding='utf-8-sig') # Excel da UTF-8

# Excel ga saqlash
df.to_excel('natija.xlsx', sheet_name='Ma'lumot', index=False)

# JSON ga saqlash
df.to_json('natija.json', orient='records', force_ascii=False)

```

3. DataFrame ni Tushunish — Dastlabki Ko'rib Chiqish

```

import pandas as pd
import numpy as np

# Titanic dataset (mashq uchun klassik dataset)
df = pd.read_csv('titanic.csv')

# — Umumiy ma'lumot —————
print(df.shape) # (891, 12) ← 891 qator, 12 ustun
print(df.ndim) # 2
print(len(df)) # 891 ← qatorlar soni
print(df.columns.tolist())
# ['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age',
#  'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']

```

```

# — Dastlabki/Oxirgi qatorlar —————
print(df.head(5))      # birinchi 5 ta
print(df.tail(5))      # oxirgi 5 ta
print(df.sample(5))    # tasodifiy 5 ta

# — Ustun turlari —————
print(df.dtypes)
# PassengerId      int64
# Survived         int64
# Pclass           int64
# Name             object ← string
# Sex              object
# Age              float64 ← NaN bor, shuning uchun float
# Fare             float64
# Cabin            object
# Embarked         object

# — Statistik xulosa —————
print(df.describe())
#      PassengerId      Survived      Pclass         Age         Fare
# count      891.000000      891.000000      891.000000      714.000000      891.000000
# mean        446.000000        0.383838        2.308642       29.699118       32.204208
# std         257.353842        0.486592        0.836071       14.526497       49.693429
# min          1.000000        0.000000        1.000000        0.420000        0.000000
# 25%         223.500000        0.000000        2.000000       20.125000        7.910400
# 50%         446.000000        0.000000        3.000000       28.000000       14.454200
# 75%         668.500000        1.000000        3.000000       38.000000       31.000000
# max         891.000000        1.000000        3.000000       80.000000      512.329200

# — To'liq ma'lumot (NaN soni ham) —————
print(df.info())
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 891 entries, 0 to 890
# Data columns (total 12 columns):
# #   Column      Non-Null Count  Dtype
# ---  -
# 0   PassengerId  891 non-null    int64
# 1   Survived     891 non-null    int64
# 2   Pclass       891 non-null    int64
# 3   Name         891 non-null    object
# 4   Sex          891 non-null    object
# 5   Age          714 non-null    float64 ← 177 ta NaN!
# 6   SibSp        891 non-null    int64
# 7   Fare         891 non-null    float64
# 8   Cabin        204 non-null    object ← 687 ta NaN!
# 9   Embarked     889 non-null    object ← 2 ta NaN

```

4. NaN Qiymatlar — Aniqlash va Tozalash

NaN (Not a Number) — yo'qolgan yoki noma'lum qiymat. ML da NaN qiymatlar model o'qitishini to'xtatib qo'yadi, shuning uchun ularni aniqlash va to'g'ri ishlash juda muhim.

NaN ni aniqlash:

```

import pandas as pd
import numpy as np

# — NaN soni —————
print(df.isnull().sum())
# PassengerId      0
# Survived         0
# Pclass           0
# Age             177  ← 177 ta yo'qolgan
# Cabin           687  ← 687 ta yo'qolgan
# Embarked         2   ← 2 ta yo'qolgan
# dtype: int64

# Foiz sifatida:
nan_foiz = (df.isnull().sum() / len(df) * 100).round(2)
print(nan_foiz[nan_foiz > 0])
# Age             19.87%
# Cabin           77.10%  ← juda ko'p!
# Embarked         0.22%

# — Qaysi qatorlarda NaN bor —————
print(df[df['Age'].isnull()].head(3))      # Age NaN bo'lgan qatorlar
print(df[df.isnull().any(axis=1)])        # istalgan ustunda NaN bor qatorlar
print(df.isnull().any(axis=1).sum())      # bunday qatorlar soni: 708

```

NaN ni to'ldirish (Imputation):

```

import pandas as pd

# — Usul 1: O'rtacha (mean) bilan — sonlar uchun —————
# Age ustunida NaN → o'rtacha yosh bilan to'ldirish
df['Age'].fillna(df['Age'].mean(), inplace=True)
# inplace=True → df ni o'zini o'zgartiradi (yangi ob'yekt yaratmaydi)

# Mediana bilan (outlier bo'lsa afzal)
df['Age_median'] = df['Age'].fillna(df['Age'].median())

# — Usul 2: Eng ko'p uchraydigan qiymat — kategoriyalar uchun
# Embarked: 'S', 'C', 'Q' — eng ko'pi bilan to'ldirish
eng_kop = df['Embarked'].mode()[0]  # 'S'
df['Embarked'].fillna(eng_kop, inplace=True)

# — Usul 3: Maxsus qiymat bilan —————
# Cabin ko'p NaN → "Unknown" deb belgilash
df['Cabin'].fillna('Unknown', inplace=True)

# — Usul 4: Oldindagi/keyindagi qiymat —————
# Vaqt qatori ma'lumlarda:
df['temperatura'].fillna(method='ffill', inplace=True)  # oldingisi bilan
df['temperatura'].fillna(method='bfill', inplace=True)  # keyingisi bilan

# — Usul 5: Interpolatsiya —————
df['narx'].interpolate(method='linear', inplace=True)

# — Usul 6: Guruh bo'yicha o'rtacha —————
# Har sinf uchun alohida o'rtacha yosh (aniqroq usul)

```

```

df['Age'] = df.groupby('Pclass')['Age'].transform(
    lambda x: x.fillna(x.median())
)
# 1-sinf: median=37, 2-sinf: median=29, 3-sinf: median=24
# Har sinf NaN → o'sha sinf mediasi bilan

# — NaN qatorlarini o'chirish —————
df_tozalangan = df.dropna() # istalgan NaN → o'chir
df_tozalangan = df.dropna(subset=['Age', 'Fare']) # faqat bu ustunlar bo'sh bo'lsa
df_tozalangan = df.dropna(thresh=10) # kamida 10 ta to'liq qiymat
bo'lsin

```

NaN bilan ishlash strategiyasi:

NaN foizi < 5% → O'rtacha/mediana/mode bilan to'ldirish
 NaN foizi 5-30% → Guruh bo'yicha imputation (Group mean/median)
 NaN foizi > 30% → Ustunni o'chirish yoki "NaN_flag" ustun qo'shish
 NaN foizi > 70% → Ustunni o'chirish (Cabin: 77% → o'chirish)

Kategorik NaN:

Kam kategoriya → Mode bilan

Ko'p NaN → "Unknown" deb belgilash

Vaqt qatorlari NaN:

ffill / bfill / interpolate

```

# — "NaN bo'lganmi?" belgisi qo'shish —————
# Bu ML uchun muhim: NaN ning o'zi ma'lumot berishi mumkin
df['Cabin_mavjud'] = df['Cabin'].notnull().astype(int)
# 0 → Cabin yo'q (ehtimol uchinchi sinf yoki kam to'lagan)
# 1 → Cabin bor (birinchi yoki ikkinchi sinf)

# — Butun ustunni o'chirish —————
df_tozalangan = df.drop(columns=['Cabin']) # bitta ustun
df_tozalangan = df.drop(columns=['Cabin', 'Ticket']) # bir nechta

```

5. Ustunlar va Qatorlar bilan Ishlash

```
import pandas as pd
```

```

# — Ustunni olish —————
ism_ustun = df['ism'] # Series
ism_ustun = df.ism # xuddi shunday (nom bo'sh joy bo'lmasa)
kop_ustun = df[['ism', 'maosh']] # DataFrame (ikki qavs!)

# — Yangi ustun qo'shish —————
df['bonus'] = df['maosh'] * 0.1
df['to'liq_maosh'] = df['maosh'] + df['bonus']
df['katta_mi'] = (df['yosh'] > 30).astype(int) # 0 yoki 1
df['ism_uzunligi'] = df['ism'].str.len()

# — Ustun nomini o'zgartirish —————
df.rename(columns={'ism': 'full_name', 'yosh': 'age'}, inplace=True)
df.columns = ['a', 'b', 'c', 'd'] # barchani o'zgartirish

```

```

# — Ustun turini o'zgartirish —————
df['yosh'] = df['yosh'].astype(int)
df['maosh'] = df['maosh'].astype(float)
df['sana'] = pd.to_datetime(df['sana']) # satrdan datetime
df['tur'] = df['tur'].astype('category') # xotira tejash

# — Indeks bilan ishlash —————
df.set_index('id', inplace=True) # 'id' ustunini indeks qil
df.reset_index(inplace=True) # indeksni ustun ga qaytarish

# — Qatorni olish: loc va iloc —————
print(df.loc[0]) # 0-indeksli qator (label)
print(df.loc[0:3, 'ism':'maosh']) # qatorlar va ustunlar (label)
print(df.iloc[0]) # 0-pozitsiyali qator (raqam)
print(df.iloc[0:3, 1:3]) # qatorlar va ustunlar (raqam)
print(df.loc[df['yosh'] > 30]) # shartli qatorlar

```

6. Filtrlash — Shartli Ma'lumot Tanlash

```
import pandas as pd
```

```

# — Oddiy shartlar —————
kattalar = df[df['yosh'] > 30]
toshkent = df[df['shahar'] == 'Toshkent']
yuqori = df[df['maosh'] >= 7000]

# — Murakkab shartlar —————
# & (va), | (yoki), ~ (inkor) – har shart qavs ichida!
toshkent_katta = df[(df['shahar'] == 'Toshkent') & (df['yosh'] > 28)]
ish_yoq = df[(df['maosh'] < 3000) | (df['maosh'].isnull())]
toshkent_emas = df[~(df['shahar'] == 'Toshkent')] # Toshkentdan tashqari

# — .isin() – ro'yxatdagi qiymatlar —————
shaharlari = ['Toshkent', 'Samarqand']
ikki_shahar = df[df['shahar'].isin(shaharlari)]

# Teskari: ro'yxatda bo'lmaganlar
boshqa = df[~df['shahar'].isin(shaharlari)]

# — .between() – oraliq —————
o_rta_yosh = df[df['yosh'].between(25, 35)] # 25 ≤ yosh ≤ 35
o_rta_maosh = df[df['maosh'].between(5000, 8000)]

# — .str metodlari – satr filtri —————
ali_lar = df[df['ism'].str.startswith('A')] # A bilan boshlangan
toshkentlik = df[df['ism'].str.contains('Toshkent')] # ichida bor
kichik = df[df['ism'].str.lower() == 'ali'] # kichik harfga o'tkazib
uzun_ism = df[df['ism'].str.len() > 5] # 5 dan uzun

# — .query() – satr ko'rinishida —————
# Oddiy o'qish uchun qulay
natija = df.query("yosh > 28 and shahar == 'Toshkent'")
natija = df.query("5000 <= maosh <= 8000")

```

```

o_rtacha = 6000
natija = df.query("maosh > @o_rtacha")      # Python o'zgaruvchisi @ bilan

# — .where() – shartga mos kelmagan → NaN —————
df['maosh_filtrlangan'] = df['maosh'].where(df['maosh'] > 5000)
# 5000 dan past → NaN, boshqalari o'zgarmaydi

# — .mask() – where() ning teskari —————
df['maosh_yuqori'] = df['maosh'].mask(df['maosh'] < 5000, 0)
# 5000 dan past → 0 bilan almashtirish

```

7. Saralash va O'zgartirish

```

import pandas as pd

# — Saralash —————
df_sorted = df.sort_values('maosh', ascending=False)      # yuqoridan pastga
df_sorted = df.sort_values(['shahar', 'maosh'],
                           ascending=[True, False])      # shahar asc, maosh
desc

# — Dublikatlar —————
print(df.duplicated().sum())      # dublikat qatorlar soni
df_tozalangan = df.drop_duplicates()      # barchasini o'chirish
df_tozalangan = df.drop_duplicates(subset=['ism', 'shahar']) # faqat bu ustunlar

# — apply() – har elementga funksiya qo'llash —————
# Ustun bo'yicha:
df['yosh_guruh'] = df['yosh'].apply(
    lambda x: 'Yosh' if x < 30 else 'O\'rta' if x < 45 else 'Katta'
)

# DataFrame bo'yicha (murakkab hisoblash):
def bonus_hisoblash(qator):
    if qator['yosh'] > 30 and qator['maosh'] > 6000:
        return qator['maosh'] * 0.15
    return qator['maosh'] * 0.05

df['bonus'] = df.apply(bonus_hisoblash, axis=1)      # axis=1 → har qator

# — map() – bitta ustunni kamaytirish —————
shahar_kod = {'Toshkent': 1, 'Samarqand': 2, 'Buxoro': 3}
df['shahar_id'] = df['shahar'].map(shahar_kod)

# — replace() – qiymatlarni almashtirish —————
df['jinsi'] = df['jinsi'].replace({'male': 'erkak', 'female': 'ayol'})

# — Ustunlarni birlashtirish —————
df['to\'liq_manzil'] = df['shahar'] + ', ' + df['tuman']

# — Satr ustunlari: str metodlari —————
df['ism_kichik'] = df['ism'].str.lower()
df['ism_katta'] = df['ism'].str.upper()
df['ism_trim'] = df['ism'].str.strip()      # bo'sh joylarni olib tashlash
df[['ism', 'familiya']] = df['to\'liq_ism'].str.split(' ', expand=True)

```

8. Groupby va Agregatsiya

Groupby — ma'lumotlarni biror ustun bo'yicha guruhlab, har guruh uchun alohida hisoblash. SQL dagi `GROUP BY` bilan bir xil mantiq.

GroupBy ishlash mexanizmi (Split-Apply-Combine):

SPLIT → Ma'lumotni guruhlariga bo'lish
APPLY → Har guruhga funksiya qo'llash
COMBINE → Natijalarni birlashtirish

Misol: `df.groupby('shahar')['maosh'].mean()`

SPLIT: Toshkent: [5000, 6100, 5800]
 Samarqand: [7200]
 Buxoro: [8500]

APPLY: Toshkent: mean([5000, 6100, 5800]) = 5633.3
 Samarqand: mean([7200]) = 7200.0
 Buxoro: mean([8500]) = 8500.0

COMBINE: shahar maosh
 Buxoro 8500.0
 Samarqand 7200.0
 Toshkent 5633.3

Asosiy agregatsiya funksiyalari:

```
import pandas as pd
```

```
# — Bitta ustun bo'yicha guruhlash
```

```
guruh = df.groupby('shahar')['maosh']
```

```
print(guruh.mean())     # o'rtacha maosh har shahar uchun
print(guruh.median())   # mediana
print(guruh.sum())      # jami maosh
print(guruh.min())      # minimal maosh
print(guruh.max())      # maksimal maosh
print(guruh.count())    # xodimlar soni
print(guruh.std())      # standart og'ish
print(guruh.var())      # dispersiya
```

```
# — Bir nechta ustun bo'yicha guruhlash
```

```
df.groupby(['shahar', 'jinsi']]['maosh'].mean()
```

```
# shahar        jinsi
# Buxoro        ayol        8200.0
#                erkak       8700.0
# Samarqand    erkak       7200.0
# Toshkent     ayol        5650.0
#                erkak       5616.7
```

```
# — Bir nechta ustun agregatsiyasi
```

```
df.groupby('shahar')[['maosh', 'yosh']].mean()
```

```
#                maosh        yosh
```

```
# shahar
# Buxoro      8500.0    35.0
# Samarqand   7200.0    32.0
# Toshkent    5633.3    27.3
```

.agg() — ko'p funksiya birgalikda:

```
import pandas as pd
```

```
# — Bir ustun, ko'p funksiya —————
natija = df.groupby('shahar')['maosh'].agg(['mean', 'min', 'max', 'count'])
print(natija)
```

```
#          mean    min    max    count
# shahar
# Buxoro      8500.0  8500  8500         1
# Samarqand   7200.0  7200  7200         1
# Toshkent    5633.3  5000  6100         3
```

```
# — Har ustun uchun boshqa funksiya —————
natija = df.groupby('shahar').agg({
    'maosh': ['mean', 'max', 'min'],
    'yosh' : ['mean', 'count'],
    'ism'   : 'count'
})
```

```
print(natija)
#          maosh          yosh          ism
#          mean    max    min    mean count  count
# shahar
# Buxoro      8500.0  8500  8500   35.0     1       1
# Samarqand  7200.0  7200  7200   32.0     1       1
# Toshkent   5633.3  6100  5000   27.3     3       3
```

```
# — Custom funksiya bilan —————
def oraliq(x):
    return x.max() - x.min()
```

```
natija = df.groupby('shahar')['maosh'].agg(
    o_rtacha='mean',
    maksimal='max',
    minimal='min',
    oraliq=oraliq          # custom funksiya
)
print(natija)
```

```
# — transform() — grupni asl o'lchamda qaytarish —————
# Har xodimning maoshi o'z shahar o'rtachasidan farqi
df['shahar_ortacha'] = df.groupby('shahar')['maosh'].transform('mean')
df['shahar_dan_farqi'] = df['maosh'] - df['shahar_ortacha']
```

```
print(df[['ism', 'shahar', 'maosh', 'shahar_ortacha', 'shahar_dan_farqi']])
#          ism    shahar    maosh    shahar_ortacha    shahar_dan_farqi
# 0      Ali  Toshkent    5000         5633.3         -633.3
# 1     Vali  Samarqand    7200         7200.0           0.0
# 2    Hasan  Toshkent    6100         5633.3         466.7
# 3  Zulfiya   Buxoro    8500         8500.0           0.0
# 4   Malika  Toshkent    5800         5633.3         166.7
```

Pivot Table:

```
import pandas as pd
```

```
# — Oddiy pivot table
```

```
jadval = pd.pivot_table(  
    df,  
    values='maosh',           # agregatsiya qilinuvchi ustun  
    index='shahar',          # qatorlar  
    columns='jinsi',         # ustunlar  
    aggfunc='mean',          # funksiya  
    fill_value=0              # NaN o'rniga  
)
```

```
print(jadval)
```

```
# jinsi      ayol      erkak  
# shahar  
# Buxoro      8200.0      8700.0  
# Samarqand      0.0      7200.0  
# Toshkent      5650.0      5616.7
```

```
# — Ko'p qiymat
```

```
jadval = pd.pivot_table(  
    df,  
    values=['maosh', 'yosh'],  
    index='shahar',  
    columns='jinsi',  
    aggfunc={'maosh': 'mean', 'yosh': 'count'}  
)
```

9. Merge va Concatenate — Jadvallarni Birlashtirish

```
import pandas as pd
```

```
# — pd.concat() — ustma-ust qo'yish
```

```
df1 = pd.DataFrame({'ism': ['Ali', 'Vali'], 'yosh': [25, 32]})  
df2 = pd.DataFrame({'ism': ['Hasan', 'Zulfiya'], 'yosh': [28, 35]})
```

```
barchasi = pd.concat([df1, df2], ignore_index=True)
```

```
print(barchasi)
```

```
#      ism  yosh  
# 0    Ali    25  
# 1    Vali    32  
# 2  Hasan    28  
# 3 Zulfiya   35
```

```
# — pd.merge() — SQL JOIN kabi
```

```
xodimlar = pd.DataFrame({  
    'id':      [1, 2, 3, 4],  
    'ism':     ['Ali', 'Vali', 'Hasan', 'Zulfiya'],  
    'bo'lim_id': [10, 20, 10, 30]  
})
```

```
bo'limlar = pd.DataFrame({  
    'bo'lim_id': [10, 20, 30],  
    'bo'lim':    ['IT', 'Moliya', 'HR']  
})
```

```

})

# INNER JOIN – faqat ikkisida ham bor
natija = pd.merge(xodimlar, bo'limlar, on='bo'lim_id', how='inner')
print(natija)
#   id      ism  bo'lim_id  bo'lim
# 0   1     Ali        10      IT
# 1   3   Hasan        10      IT
# 2   2     Vali        20  Moliya
# 3   4  Zulfiya        30      HR

# LEFT JOIN – chapda bor, o'ngda yo'q → NaN
natija = pd.merge(xodimlar, bo'limlar, on='bo'lim_id', how='left')

# Turli ustun nomlari bilan
natija = pd.merge(
    xodimlar, bo'limlar,
    left_on='bo'lim_id',      # chapda
    right_on='bo'lim_id',    # o'ngda
    how='inner'
)

```

10. Real Dataset — To'liq Amaliy Misol

```

import pandas as pd
import numpy as np

# =====
# Vazifa: Titanic datasetini tahlil qilish
# Savollar:
# 1. Omon qolish ehtimoli jinsi va sinfga qarab qanday farqlanadi?
# 2. Yosh qaysi sinfdan yuqori?
# 3. Eng yuqori narx to'lagan 10 ta yo'lovchi kimlar?
# =====

# — 1. YUKLASH —=====
url = ('https://raw.githubusercontent.com/datasciencedojo/'
      'datasets/master/titanic.csv')
df = pd.read_csv(url)
print(f"O'lcham: {df.shape}")          # (891, 12)

# — 2. DASTLABKI TEKSHIRISH —=====
print("\nNaN qiymatlari:")
print(df.isnull().sum()[df.isnull().sum() > 0])
# Age          177
# Cabin         687
# Embarked       2

# — 3. TOZALASH —=====
# Cabin: 77% NaN → belgiga aylantirish
df['Cabin_mavjud'] = df['Cabin'].notnull().astype(int)
df.drop(columns=['Cabin'], inplace=True)

# Age: sinf bo'yicha mediana bilan to'ldirish
df['Age'] = df.groupby('Pclass')['Age'].transform(

```

```

        lambda x: x.fillna(x.median())
    )

# Embarked: mode bilan to'ldirish
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

# Keraksiz ustunlar
df.drop(columns=['PassengerId', 'Name', 'Ticket'], inplace=True)

print(f"\nTozalangandan keyin NaN: {df.isnull().sum().sum()}") # 0

# — 4. YANGI USTUNLAR —————
# Oila a'zolari soni
df['Oila'] = df['SibSp'] + df['Parch'] + 1 # o'zi + boshqalar

# Yosh guruhi
def yosh_guruh(yosh):
    if yosh < 13: return 'Bola'
    elif yosh < 18: return 'O\'smir'
    elif yosh < 60: return 'Katta'
    else: return 'Keksa'

df['Yosh_guruh'] = df['Age'].apply(yosh_guruh)

# — 5. TAHLIL: JINS VA SINIF BO'YICHA OMON QOLISH —————
print("\n=== Jins bo'yicha omon qolish ===")
jins_natija = df.groupby('Sex')['Survived'].agg(
    omon_qoldi='sum',
    jami='count',
    ehtimol=lambda x: f"{x.mean()*100:.1f}%"
)
print(jins_natija)
#           omon_qoldi  jami ehtimol
# Sex
# female           233   314   74.2%
# male             109   577   18.9%
# → Ayollar 74.2%, Erkaklar faqat 18.9% omon qoldi!

print("\n=== Sinf va jins bo'yicha omon qolish ===")
sinf_jins = df.groupby(['Pclass', 'Sex'])['Survived'].mean().unstack()
sinf_jins = (sinf_jins * 100).round(1)
sinf_jins.columns.name = None
print(sinf_jins)
#           female   male
# Pclass
# 1           96.8   36.9   ← 1-sinf ayollar: deyarli barchasi!
# 2           92.1   15.7
# 3           50.0   13.5   ← 3-sinf erkaklar: 13.5% faqat

# — 6. TAHLIL: YOSH —————
print("\n=== Sinf bo'yicha o'rtacha yosh ===")
yosh_tahlil = df.groupby('Pclass')['Age'].agg(
    ortacha=lambda x: round(x.mean(), 1),
    mediana='median',
    minimal='min',
    maksimal='max'
)

```

```

print(yosh_tahlil)
#          ortacha  mediana  minimal  maksimal
# Pclass
# 1          38.2    37.0    0.92    80.0
# 2          29.9    29.0    0.67    70.0
# 3          25.1    24.0    0.42    74.0

# — 7. TAHLIL: NARX —————
print("\n=== Eng qimmat 10 ta chipta ===")
top10 = (df.nlargest(10, 'Fare')
         [['Sex', 'Age', 'Pclass', 'Fare', 'Survived']]
         .reset_index(drop=True))
print(top10)

# — 8. YOSH GURUHI BO'YICHA OMON QOLISH —————
print("\n=== Yosh guruhi bo'yicha omon qolish ===")
tartib = ['Bola', 'O'smir', 'Katta', 'Keksa']
yosh_natija = (df.groupby('Yosh_guruh')['Survived']
               .mean()
               .reindex(tartib)
               .mul(100)
               .round(1))
print(yosh_natija)
# Yosh_guruh
# Bola      53.98%  ← bolalar ko'proq omon qoldi
# O'smir    42.86%
# Katta     37.89%
# Keksa     22.73%  ← keksalar eng kam

# — 9. NATIJANI SAQLASH —————
df.to_csv('titanic_tozalangan.csv', index=False)
sinf_jins.to_csv('sinf_jins_tahlil.csv')
print("\nFayllar saqlandi!")

```

11. Foydali Pandas Metodlari — Qisqa Ma'lumotnoma

```
import pandas as pd
```

```

# — Qiymatlar haqida —————
df['shahar'].value_counts()          # har qiymat necha marta
df['shahar'].value_counts(normalize=True) # foiz sifatida
df['maosh'].unique()                 # noyob qiymatlar
df['maosh'].nunique()                 # noyob qiymatlar soni

# — Korrelyatsiya —————
df.corr()                            # barcha son ustunlar korrelyatsiyasi
df['yosh'].corr(df['maosh'])           # ikki ustun orasida
df.corr()['Survived'].sort_values()   # Survived bilan korrelyatsiya

# — Ustunni guruhlarga bo'lish —————
df['yosh_kategori'] = pd.cut(
    df['yosh'],
    bins=[0, 18, 35, 60, 100],
    labels=['Yosh', 'O\'rta', 'Balog\'at', 'Keksa']
)

```

```
df['maosh_kvartil'] = pd.qcut(
    df['maosh'],
    q=4,
    labels=['Past', 'O\'rta-past', 'O\'rta-yuqori', 'Yuqori']
)
```

— Vaqt bilan ishlash

```
df['sana'] = pd.to_datetime(df['sana'])
df['yil'] = df['sana'].dt.year
df['oy'] = df['sana'].dt.month
df['kun'] = df['sana'].dt.day
df['hafta_kuni'] = df['sana'].dt.day_name()
```

12. Xulosa

PANDAS DATAFRAME ASOSIY OPERATSIYALARI:

YUKLASH:

```
pd.read_csv()    → CSV fayl
pd.read_excel()  → Excel fayl
pd.read_sql()    → SQL jadval
df.to_csv()      → Saqlash
```

NaN TOZALASH:

```
df.isnull().sum()      → NaN soni
df.fillna(mean)        → O'rtacha bilan
df.fillna(mode()[0])   → Mode bilan
groupby().transform(fill) → Guruh bo'yicha (eng aniq)
df.dropna()            → Qatorni o'chirish
NaN > 70%              → Ustunni o'chirish
```

FILTRLASH:

```
df[df['yosh'] > 30]      → Oddiy shart
df[(shart1) & (shart2)]  → Murakkab (qavs!)
df[df['shahar'].isin(['T','S'])] → Ro'yxat
df.query("yosh > 30 and maosh > 5000") → Satr ko'rinishi
```

GROUPBY + AGREGATSIYA:

```
.groupby('ustun')['qiymat'].mean() → O'rtacha
.agg({'ustun': ['min','max','mean']}) → Bir nechta
.transform('mean') → Guruh natijasini asl shakl
```

MERGE:

```
pd.concat([df1,df2])      → Ustma-ust
pd.merge(df1, df2, on='id',
        how='inner/left/right') → SQL JOIN
```

ML UCHUN MUHIM:

```
df.values / df.to_numpy() → NumPy array (sklearn uchun)
pd.get_dummies(df['tur']) → One-Hot encoding
df.corr()                → Feature tanlash uchun
groupby().transform()    → Guruh bo'yicha imputation
```

💡 **Eslab qol:** `groupby()` dan keyin `mean()` yozganingda indeks o'zgaradi — `reset_index()` chaqir! `fillna()` da `inplace=True` yozmasa o'zgarish saqlanmaydi.

ML uchun eng muhim qoida: **NaN ni to'ldirish parametrlarini (mean, std) faqat train datadan hisoblash kerak** — test dataga train parametrlarini qo'llash, aks holda data leakage bo'ladi!

SAVOL 19:

MA'LUMOTLARNI MASHINALI O'QITISHGA TAYYORLASH BOSQICHLARI: YO'QOLGAN QIYMATLARNI TO'LDIRISH, KATEGORIK O'ZGARUVCHILARNI KODLASH (LABEL ENCODING , ONE-HOT ENCODING), NORMALIZATSIYA VA STANDARTIZATSIYA.

1. Nima Uchun Tayyorlash Kerak?

Haqiqiy dunyo ma'lumotlari hech qachon "tayyor" bo'lmaydi. ML modellari esa faqat **son qiymatlarni, bir xil o'lchamdagi va to'liq** ma'lumotlarni qabul qiladi.

Haqiqiy ma'lumot:

ism	yosh	shahar	maosh
Ali	25	Toshkent	5000
Vali	NaN	Buxoro	7200
?	32	Toshkent	NaN
Hasan	28	NaN	6100

ML modeli kutgani:

faqat sonlar, NaN yo'q,
bir xil o'lcham

$X = \begin{bmatrix} 0.12, 0, 1, 0.43 \\ 0.67, 1, 0, 0.88 \\ 0.34, 0, 1, 0.61 \end{bmatrix}$

↑ string, NaN, aralash o'lcham → model qabul qilmaydi!

Tayyorlash bosqichlari ketma-ketligi:

1. Yo'qolgan qiymatlarni to'ldirish (NaN → son yoki kategoriya)
↓
 2. Kategorik o'zgaruvchilarni kodlash (string → son)
↓
 3. Normalizatsiya / Standartizatsiya (katta sonlar → kichik oraliq)
↓
 4. Feature selection / engineering (muhim ustunlarni tanlash)
↓
- ML modeli

2. Yo'qolgan Qiymatlarni To'ldirish (Imputation)

Nima uchun NaN paydo bo'ladi?

Sabab

Misol

Anketada javob berilmagan

Daromad ustuni bo'sh qoldirilgan

Sensor vaqtincha ishlamagan	Harorat o'lchagich 2 soat o'chgan
Ma'lumot birlashtirishda	Ikki jadval JOIN da mos kelmagan
Maxfiy saqlanib, o'chirilgan	Kasallik tarixi oshkor qilinmagan

To'ldirish strategiyalari:

NaN TO'LDIRISH STRATEGIYASI		
NaN foizi < 5%	NaN foizi 5% - 30%	NaN > 30%
Oddiy to'ldirish mean/median/mode	Guruh bo'yicha to'ldirish (Group imputation)	Ustunni o'chirish YOKI "mavjud" belgisi

Son ustunlar uchun:

O'rtacha (Mean):

Qachon: normal taqsimotda, outlier yo'q

Muammo: outlier bo'lsa o'rtacha siljiydi

Misol: [10, 12, 11, NaN, 13] → NaN = 11.5

Mediana (Median):

Qachon: outlier bor yoki taqsimot qiyshiq

Afzallik: outlier ta'sir qilmaydi

Misol: [10, 12, 11, NaN, 100] → NaN = 11 (median)
mean = 33.25 ← noto'g'ri bo'lardi!

Guruh bo'yicha mediana (BEST PRACTICE):

Qachon: guruhlar o'rtasida farq katta

Misol: yosh NaN → o'sha sinf (1,2,3) mediasi bilan

→ 1-sinf median yosh: 38, 3-sinf: 24

→ Barcha uchun 29 yoz deyish noto'g'ri!

Kategorik ustunlar uchun:

Mode (eng ko'p uchraydigan):

Qachon: NaN kam (< 5%)

Misol: ['S','S','C','Q','S',NaN] → NaN = 'S'

"Unknown" / "Other" kategoriya:

Qachon: NaN ko'p (> 10%)

Sabab: NaN ning o'zi ma'lumot berishi mumkin

Misol: Cabin bo'sh → uchinchi sinf belgisi

NaN belgisi qo'shish:

Qachon: NaN tasodifiy emas, ma'noli

Misol: df['Cabin_mavjud'] = df['Cabin'].notnull().astype(int)

→ NaN ning o'zi "kambag'al yo'lovchi" degan signal

```
from sklearn.impute import SimpleImputer
import pandas as pd
```

```
# Son ustunlar → median
```

```
num_imputer = SimpleImputer(strategy='median')
df[['Age', 'Fare']] = num_imputer.fit_transform(df[['Age', 'Fare']])

# Kategorik ustunlar → mode
cat_imputer = SimpleImputer(strategy='most_frequent')
df[['Embarked']] = cat_imputer.fit_transform(df[['Embarked']])
```

3. Kategorik O'zgaruvchilarni Kodlash

ML modellari **faqat son** qabul qiladi. "Toshkent", "erkak", "yuqori" kabi satrlar bevosita modelga berilmaydi — ularni sonlarga aylantirish shart.

Kategorik o'zgaruvchilarning 2 turi:

NOMINAL (tartibsiz):
Qiymatlar o'rtasida
matematik tartib yo'q

Misol:
shahar: Toshkent, Buxoro
rang: qizil, ko'k, yashil
qon guruhi: A, B, AB, O

ORDINAL (tartibli):
Qiymatlar o'rtasida
aniq tartib bor

Misol:
ta'lim: maktab < bakalavr < magistr
baho: yomon < o'rta < yaxshi < a'lo
hajm: S < M < L < XL

→ One-Hot Encoding ishlatiladi → Label/Ordinal Encoding ishlatiladi

4. Label Encoding — Tartibli Kodlash

Ta'rif: Har bir kategoriyaga bitta butun son beriladi: 0, 1, 2, 3...

Asl qiymat:	Kod:
"maktab" →	0
"bakalavr" →	1
"magistr" →	2
"doktorant" →	3

Qachon to'g'ri ishlatiladi?

✓ TO'G'RI: Ordinal kategoriyalar (tartib muhim)
ta'lim darajasi: 0 < 1 < 2 < 3 ← mantiqiy!
mijoz sathi: bronze < silver < gold < platinum
kasallik og'irligi: engil < o'rta < og'ir

✗ NOTO'G'RI: Nominal kategoriyalar (tartib yo'q)
shahar: Toshkent=0, Buxoro=1, Samarqand=2
Model "Samarqand > Toshkent" deb o'ylaydi – XATO!
rang: qizil=0, ko'k=1, yashil=2
Model "yashil = qizil + ko'k" deb o'ylaydi – XATO!

Raqamli tartib muammosi:

Misol: 3 ta shahar → Label Encoding

Toshkent = 0
Buxoro = 1
Samarqand = 2

Linear Regression bu qiymatlarni shunday ko'radi:
|Toshkent - Buxoro| = 1
|Buxoro - Samarqand| = 1
|Toshkent - Samarqand| = 2 ← Samarqand "ikki baravar uzoq"

Bu matematik xulosa NOTO'G'RI – shaharlar o'rtasida
bunday munosabat yo'q!

```
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder
```

```
# LabelEncoder – bitta ustun uchun
```

```
le = LabelEncoder()  
df['ta_lim_kod'] = le.fit_transform(df['ta_lim'])  
# ['maktab', 'magistr', 'bakalavr', 'maktab'] → [2, 3, 0, 2]  
# Tartib alfavit bo'yicha – MUAMMO!
```

```
# OrdinalEncoder – tartibni o'zing belgilaysan (TO'G'RI USUL)
```

```
oe = OrdinalEncoder(categories=[['maktab', 'bakalavr', 'magistr', 'doktorant']])  
df['ta_lim_kod'] = oe.fit_transform(df[['ta_lim']])  
# maktab=0, bakalavr=1, magistr=2, doktorant=3 ← siz belgiladingiz
```

5. One-Hot Encoding — Nominal Kodlash

Ta'rif: Har bir kategoriya uchun **alohida ustun** yaratiladi. O'sha kategoriyaga mos ustunda **1**, qolganlarida **0** yoziladi.

Asl qiymat:		One-Hot Encoding:		
shahar		shahar_Toshkent	shahar_Buxoro	shahar_Samarqand
Toshkent	→	1	0	0
Buxoro	→	0	1	0
Samarqand	→	0	0	1
Toshkent	→	1	0	0

Nima uchun afzal? Modelga "Toshkent", "Buxoro", "Samarqand" o'rtasida **hech qanday matematik munosabat yo'q** degan signal beriladi. **1** va **0** dan boshqa son yo'q.

Dummy Variable Trap:

MUAMMO: 3 kategoriya → 3 ustun yaratilsa

shahar_Toshkent	shahar_Buxoro	shahar_Samarqand	
1	0	0	← Toshkent
0	1	0	← Buxoro
0	0	1	← Samarqand

shahar_Toshkent = 1 - shahar_Buxoro - shahar_Samarqand
 → Birinchi ustun boshqalardan hisoblanadi!
 → Multicollinearity (ko'p ustunlar bir-biriga bog'liq)
 → Linear modellar uchun muammo

YECHIM: n-1 ustun yaratish (drop_first=True)

shahar_Buxoro	shahar_Samarqand	
0	0	← Toshkent (ikkalasi 0 bo'lsa)
1	0	← Buxoro
0	1	← Samarqand

```
import pandas as pd
```

pd.get_dummies – eng oddiy usul

```
df_encoded = pd.get_dummies(df, columns=['shahar'], drop_first=True)
```

drop_first=True → Dummy trap dan himoya

sklearn OneHotEncoder – Pipeline uchun qulay

```
from sklearn.preprocessing import OneHotEncoder
```

```
ohe = OneHotEncoder(drop='first', sparse_output=False)
```

Label Encoding vs One-Hot Encoding:

Xususiyat	Label Encoding	One-Hot Encoding
Kategoriya turi	Ordinal (tartibli)	Nominal (tartibsiz)
Yangi ustunlar	Yo'q (1 ustun)	n-1 ta yangi ustun
Xotira	Kam	Ko'p (ko'p kategoriya)
Math munosabat	Bor ($0 < 1 < 2$)	Yo'q (faqat 0 va 1)
Algoritmlar	Tree-based (RF, XGBoost)	Linear, SVM, NN
Ko'p kategoriya	Muammo yo'q	Muammo: 100+ ustun
Misol	ta'lim, hajm, baho	shahar, rang, qon guruhi

Ko'p kategoriyali ustun muammosi:

Masalan: "shahar" ustunida 500 ta noyob shahar

One-Hot Encoding → 499 ta yangi ustun!

→ Katta xotira sarfi

→ Kursivlik (sparsity) – 499 ustundan 498 tasi 0

→ Model o'qitish sekinlashadi

Yechimlar:

1. Target Encoding → har kategoriyaga target o'rtachasi

2. Frequency Encoding → har kategoriya uchun uchrashuv chastotasi

3. Embedding → neyron tarmoqlarda (Word2Vec kabi)

4. Rare kategoriyalar → kamdan uchraydigan → "Other" ga birlashtirish

6. Normalizatsiya — Min-Max Scaling

Ta'rif: Barcha qiymatlarni `[0, 1]` oralig'iga o'tkazish.

Formula: $X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$

Misol:

Asl: `[200, 400, 600, 800, 1000]`
min=200, max=1000

200 → $(200-200)/(1000-200) = 0.000$
400 → $(400-200)/(1000-200) = 0.250$
600 → $(600-200)/(1000-200) = 0.500$
800 → $(800-200)/(1000-200) = 0.750$
1000 → $(1000-200)/(1000-200) = 1.000$

Norm: `[0.0, 0.25, 0.50, 0.75, 1.0]`

Normalizatsiyaning ishlash mexanizmi:

Nima uchun kerak?

yosh : `[20, 30, 25, 45]` → o'rtacha ~30, oraliq 20-45
maosh : `[500, 5000, 2000, 8000]` → o'rtacha ~3875, oraliq 500-8000

Model masofani hisoblaydi: $\sqrt{(yosh_1 - yosh_2)^2 + (maosh_1 - maosh_2)^2}$
maosh farqi = 4500, yosh farqi = 10
→ maosh 450x kuchli ta'sir qiladi!
→ model "maosh muhimroq" deb o'ylaydi – NOTO'G'RI

Normalizatsiyadan keyin:

yosh : `[0.0, 0.40, 0.20, 1.0]`
maosh : `[0.0, 0.60, 0.20, 1.0]`
→ Ikkisi teng ta'sir qiladi ✓

Normalizatsiyaning zaif tomoni:

MUAMMO: Outlier (g'ayritabiiy qiymat) bo'lsa

Asl: `[100, 110, 105, 108, 1000]` ← 1000 outlier!
min=100, max=1000

100 → 0.000
110 → 0.011 ← 110 va 100 deyarli bir xil!
105 → 0.006
108 → 0.009
1000 → 1.000

Normal qiymatlar hammasi 0.006-0.011 oralig'ida "yig'ilib" qoldi
→ Ma'lumot zichligi yo'qoldi → model kam o'rganadi

from sklearn.preprocessing **import** MinMaxScaler

```
scaler = MinMaxScaler(feature_range=(0, 1))
X_train_norm = scaler.fit_transform(X_train)    # fit + transform
X_test_norm = scaler.transform(X_test)          # faqat transform!
# MUHIM: test data uchun train parametrlari ishlatiladi
```

7. Standartizatsiya — Z-Score Scaling

Ta'rif: Qiymatlarni o'rtachasi `0`, standart og'ishi `1` bo'ladigan taqsimotga o'tkazish.

Formula: $X_{std} = (X - \mu) / \sigma$
 μ = o'rtacha (mean), σ = standart og'ish (std)

Misol:

Asl: [20, 30, 25, 35, 40]

$\mu = 30$, $\sigma = 7.07$

20 → $(20-30)/7.07 = -1.41$ (o'rtachadan 1.41 og'ish pastda)

25 → $(25-30)/7.07 = -0.71$

30 → $(30-30)/7.07 = 0.00$ (o'rtachada)

35 → $(35-30)/7.07 = +0.71$

40 → $(40-30)/7.07 = +1.41$ (o'rtachadan 1.41 og'ish yuqorida)

Std: [-1.41, -0.71, 0.00, +0.71, +1.41]

O'rtacha = 0, Std = 1 

Standartizatsiyaning kuchi:

AFZALLIK 1: Outlier ga chidamli

Asl: [100, 110, 105, 108, 1000]

$\mu=284.6$, $\sigma=359.5$

100 → -0.513 ← normal qiymatlar

110 → -0.485 farqlanib ko'rinadi

105 → -0.499

108 → -0.491

1000 → +1.988 ← outlier uzoq, lekin boshqalarni "ezib" qo'ymadi

AFZALLIK 2: Cheksiz oraliq

[0,1] emas – salbiy ham bo'lishi mumkin

→ Normalizatsiya kabi "squash" qilmaydi

→ Gaussian (normal) taqsimot taxmini ishlaydi

from sklearn.preprocessing **import** StandardScaler

```
scaler = StandardScaler()
```

```
X_train_std = scaler.fit_transform(X_train)
```

```
X_test_std = scaler.transform(X_test)
```

```
print(scaler.mean_) # har ustun o'rtachasi (train dan)
```

```
print(scaler.scale_) # har ustun standart og'ishi (train dan)
```

8. Normalizatsiya vs Standartizatsiya

Xususiyat	Normalizatsiya (Min-Max)	Standartizatsiya (Z-Score)

Formula	$(X - \min) / (\max - \min)$	$(X - \mu) / \sigma$
Natija oraligi	$[0, 1]$	$(-\infty, +\infty), \sim [-3, 3]$
O'rtacha	O'zgarmaydi	0 bo'ladi
Standart og'ish	O'zgarmaydi	1 bo'ladi
Outlier ta'siri	Kuchli (⚠)	Kamroq (✅)
Taqsimot shakli	O'zgarmaydi	O'zgarmaydi
Test dataga	Train min/max	Train μ/σ
Qachon ishlatish ✅	Neural Network Image processing K-means clustering	Linear/Logistic Reg SVM PCA
❌ Ishlatma	$[0,1]$ oraliq kerak Outlier bo'lsa Normal taqsimot kerak	Outlier bor $[0,1]$ kerak bo'lsa
Algoritm talab qilmaydi (scaling kerak emas)	Decision Tree Random Forest XGBoost	Decision Tree Random Forest XGBoost

Qoidalar:

Outlier bor?

Ha → Standartizatsiya yoki RobustScaler

Yo'q → Normalizatsiya yoki Standartizatsiya

Algoritm nima?

KNN, SVM, Neural Net → Scaling SHART (masofa hisoblaydi)

Linear, Logistic Reg → Standartizatsiya (gradient descent)

Decision Tree, RF → Scaling SHART EMAS (chegara qiymat)

K-means → Normalizatsiya (masofa hisoblaydi)

Taqsimot normal (Gaussian)?

Ha → Standartizatsiya afzal

Yo'q → RobustScaler (quartile asosida, outlier chidamli)

9. Data Leakage — Eng Ko'p Uchraydigan Xato

Ta'rif: Test ma'lumotidagi axborot model o'qitishiga **noqonuniy kirib kelishi**. Natijada model sun'iy ravishda yuqori ko'rsatkichga ega bo'lib, haqiqiy ishlatishda yomon ishlaydi.

NOTO'G'RI usul (Data Leakage):

Barcha data → `scaler.fit_transform()` → Train/Test bo'lish

↑

Bu erda test data ham "ko'rildi"!

min, max, mean, std → test ma'lumoti kirib ketdi

TO'G'RI usul:

Barcha data → Train / Test bo'lish

↓

```

scaler.fit_transform(X_train)    ← faqat train
scaler.transform(X_test)        ← faqat transform!

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. AVVAL bo'lish
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 2. KEYIN scaling – faqat train dan fit!
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit + transform
X_test_scaled = scaler.transform(X_test)      # faqat transform

```

10. Sklearn Pipeline — To'g'ri Yondashuv

Pipeline — barcha tayyorlash bosqichlarini zanjirga bog'lash. Data leakage dan himoya va kodni tartibli saqlash uchun.

Pipeline ishlash mantiqi:

```

fit(X_train) chaqirilganda:
→ Imputer.fit(X_train) → Imputer.transform(X_train)
→ Encoder.fit(X_train) → Encoder.transform(X_train)
→ Scaler.fit(X_train) → Scaler.transform(X_train)
→ Model.fit(X_transformed)

```

```

predict(X_test) chaqirilganda:
→ Imputer.transform(X_test) ← train parametrlari!
→ Encoder.transform(X_test) ← train parametrlari!
→ Scaler.transform(X_test) ← train parametrlari!
→ Model.predict(X_transformed)

```

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

```

```

# Son va kategorik ustunlarni alohida ishlash
son_ustunlar = ['yosh', 'maosh', 'tajriba']
kat_ustunlar = ['shahar', 'lavozim']

```

```

son_transform = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

kat_transform = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(drop='first', sparse_output=False))
])

```

```
preprocessor = ColumnTransformer([
    ('son', son_transform, son_ustunlar),
    ('kat', kat_transform, kat_ustunlar)
])
```

11. Xulosa

ML GA TAYYORLASH BOSQICHLARI:

1. NaN TO'LDIRISH

Son → median (outlier bor), mean (yo'q)
Kat → mode (kam NaN), "Unknown" (ko'p NaN)
Guruh bo'yicha imputation → ENG ANIQ USUL
NaN > 70% → ustunni o'chir
NaN belgisi → `df['X_mavjud'] = df['X'].notnull().astype(int)`

2. KATEGORIK KODLASH

Ordinal (tartibli) → OrdinalEncoder (tartibni o'zing bel)
Nominal (tartibsiz) → OneHotEncoder (drop='first!')
Ko'p kategoriya → Target/Frequency Encoding
Tree-based model → LabelEncoder ham qabul qiladi

3. NORMALIZATSIYA (Min-Max)

Formula: $(X - \min) / (\max - \min) \rightarrow [0, 1]$
Qachon: Neural Net, K-means, tasvir
Muammo: outlier bo'lsa siqiladi

4. STANDARTIZATSIYA (Z-Score)

Formula: $(X - \text{mean}) / \text{std} \rightarrow \text{mean}=0, \text{std}=1$
Qachon: Linear/Logistic Reg, SVM, PCA
Afzallik: outlier ga chidamliroq

5. DATA LEAKAGE OLDINI OLISH

AVVAL train/test bo'l
KEYIN `scaler.fit(X_train)` — faqat train!
`X_test` → faqat `.transform()` — hech qachon `.fit_transform()`!

6. PIPELINE

Barcha bosqichlarni zanjirga bog'lash
Data leakage avtomatik oldini olinadi
Cross-validation bilan ishlaydi

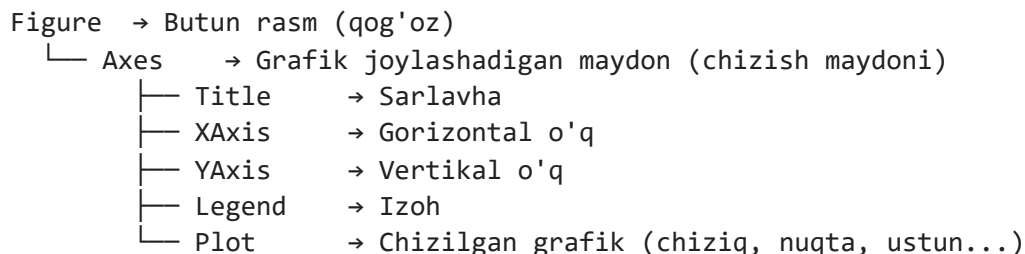
💡 **Eslab qol:** Decision Tree, Random Forest, XGBoost kabi tree-based algoritmlar uchun **scaling shart emas** — ular chegara qiymatlar bo'yicha bo'ladi, masofani hisoblamas. Lekin KNN, SVM, Linear Regression, Neural Network uchun scaling **majburiy** — aks holda katta sonli ustun modelni to'liq boshqaradi!

● SAVOL 20:

MATPLOTLIB YORDAMIDA CHIZIQLI GRAFIK, GISTOGRAMMA VA SCATTER PLOT YARATISH.
GRAFIK ELEMENTLARINI SOZLASH: SARLAVHA, O'Q NOMLARI, RANGLAR, LEGEND VA GRAFIKNI

1. Matplotlib Arxitekturası — Nazariy Asos

Matplotlib ni tushunish uchun uning **qatlam tuzilishini** bilish kerak:



Muhim farq:

```
plt.plot() → "pyplot" interfeysi – oddiy, qisqa
ax.plot() → "Axes" interfeysi – kuchli, professional
```

```
Kichik skript → plt.* yetarli
Katta loyiha → ax.* tavsiya etiladi
```

2. Figure va Axes — Asosiy Tushunchalar

```
import matplotlib.pyplot as plt
```

Usul 1 – pyplot (sodda):

```
plt.plot([1, 2, 3], [4, 5, 6])
plt.show()
```

Usul 2 – Figure/Axes (professional):

```
fig, ax = plt.subplots(figsize=(10, 6))
# figsize → dyuym: (eni, balandligi)
# fig → butun rasm ob'ekti
# ax → chizish maydoni ob'ekti
```

```
ax.plot([1, 2, 3], [4, 5, 6])
plt.show()
```

Bir sahifada bir necha grafik:

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
# 2 qator, 3 ustun = 6 ta grafik
# axes[0][0] → birinchi, axes[1][2] → oxirgi
```

3. Chiziqli Grafik (Line Plot)

Qachon ishlatiladi: Vaqt o'tishi bilan o'zgarishni ko'rsatish — temperatura, narx, foydalanuvchilar soni.

Nazariy asos: X o'qi ko'pincha **vaqt** yoki **ketma-ket son**, Y o'qi esa **qiymat**. Nuqtalar chiziq bilan bog'lanadi — bu **tendensiya** ni ko'rsatadi.

```
fig, ax = plt.subplots(figsize=(10, 5))
```

```
ax.plot(
    x, y,
    color="steelblue",      # rang
    linewidth=2,            # chiziq qalinligi
    linestyle="--",        # chiziq turi: '-', '--', '-.', ':'
    marker="o",             # nuqta belgisi: 'o', 's', '^', '*', 'D'
    markersize=6,          # nuqta o'lchami
    label="Daromad"        # legend uchun nom
)
```

Linestyle turlari:

'-'	→ to'liq chiziq	————
'--'	→ kesma chiziq	- - - -
'-.'	→ nuqta-chiziq	-.-.-.-
':'	→ nuqtali	

Marker turlari:

'o'	→ doira	's'	→ kvadrat
'^'	→ uchburchak	'D'	→ olmoc
'*'	→ yulduz	'+'	→ xoch

4. Gistogramma (Histogram)

Qachon ishlatiladi: Ma'lumotlarning **taqsimotini** ko'rsatish. Masalan: talabalar baholarining taqsimoti, yoshlar bo'yicha taqsimot.

Nazariy asos: Ma'lumotlar **bins** (chelaklar) ga bo'linadi. Har chelak o'sha oraliqda nechta qiymat borligini ko'rsatadi. Bins soni muhim:

Kam bins	→ juda umumiy, tafsilot yo'q
Ko'p bins	→ juda shovqinli, tendensiya ko'rinmaydi
Optimal	→ ma'lumot tuzilishini ochib beradi

```
ax.hist(
    ma_lumotlar,
    bins=30,                # chelaklar soni
    color="steelblue",
    edgecolor="white",      # ustunlar orasidagi chiziq
    alpha=0.7,              # shaffoflik (0-1)
    density=False,         # True = chastota o'rniga zichlik
    label="Taqsimot"
)
```

bins tanlash qoidasi:

Sturges qoidasi:	$\text{bins} = 1 + \log_2(n)$
$n=100$	→ $\text{bins} \approx 8$

```
n=1000 → bins≈11  
n=10000 → bins≈15
```

Yoki: `bins="auto"` — matplotlib o'zi tanlaydi

5. Scatter Plot (Tarqoq Grafik)

Qachon ishlatiladi: Ikki o'zgaruvchi orasidagi **bog'liqlikni** (korrelyatsiyani) ko'rsatish. Masalan: ta'lim va daromad, reklama xarajati va sotuvlar.

Nazariy asos: Har nuqta — bitta kuzatuv. Nuqtalar **yuqoriga o'ng** yo'nalishda — musbat korrelyatsiya, **pastga o'ng** — manfiy korrelyatsiya, tartibsiz — korrelyatsiya yo'q.

```
ax.scatter(  
    x, y,  
    c=ranglar,          # har nuqta rangi (ixtiyoriy)  
    s=o_lchamlar,       # har nuqta o'lchami  
    alpha=0.6,          # shaffoflik  
    cmap="viridis",     # rang xaritasi (c array bo'lsa)  
    edgecolors="white", # nuqta chegarasi  
    linewidths=0.5,  
    label="Kuzatuvlar"  
)
```

Korrelyatsiya vizual talqini:

Nuqtalar yuqori chap → past chap yo'nalishda: manfiy korrelyatsiya
Nuqtalar past chap → yuqori o'ng yo'nalishda: musbat korrelyatsiya
Nuqtalar tartibsiz: korrelyatsiya yo'q

6. Grafik Elementlarini Sozlash

Sarlavha va o'q nomlari:

```
# Sarlavha  
ax.set_title("Oylik Daromad Dinamikasi", fontsize=16, fontweight="bold", pad=15)  
  
# O'q nomlari  
ax.set_xlabel("Oy", fontsize=13, labelpad=10)  
ax.set_ylabel("Daromad (so'm)", fontsize=13, labelpad=10)  
  
# O'q chegaralari  
ax.set_xlim(0, 12)  
ax.set_ylim(0, 1000000)  
  
# O'q belgilari (ticks)  
ax.set_xticks([1,2,3,4,5,6,7,8,9,10,11,12])  
ax.set_xticklabels(["Yan", "Fev", "Mar", "Apr", "May", "Iyn",  
                    "Iyl", "Avg", "Sen", "Okt", "Noy", "Dek"],  
                    rotation=45) # burilish burchagi
```

Legend (Izoh):

```
ax.legend(
    loc="upper left",      # joylashuv
    fontsize=11,
    framealpha=0.9,        # legend fonining shaffoflik
    edgecolor="gray",
    title="Ko'rsatkichlar"
)
```

loc parametri:

"upper left"	"upper center"	"upper right"
"center left"	"center"	"center right"
"lower left"	"lower center"	"lower right"
"best"	← matplotlib o'zi tanlaydi (tavsiya)	

Setka (Grid):

```
ax.grid(True, linestyle="--", alpha=0.5, color="gray")
ax.set_axisbelow(True) # setka grafiklar ostida bo'lsin
```

Ranglar:

```
# Named ranglar:
"red", "blue", "green", "black", "white", "gray"
"steelblue", "tomato", "goldenrod", "mediumseagreen"
```

```
# HEX ranglar:
"#3498db", "#e74c3c", "#2ecc71", "#f39c12"
```

```
# RGB (0-1 oralig'ida):
(0.2, 0.4, 0.8)
```

```
# Rang palitralari (bir necha grafik uchun):
import matplotlib.pyplot as plt
ranglar = plt.cm.tab10.colors # 10 ta rang
```

7. Grafikni Saqlash

```
fig.savefig(
    "grafik.png",
    dpi=300,          # sifat (dots per inch)
    bbox_inches="tight", # chetlarni qirqmaslik
    facecolor="white",  # fon rangi
    format="png"        # png, pdf, svg, jpg
)
```

Format tanlash:

- .png → Veb, taqdimot – sifatli, shaffoflik bor
- .pdf → Bosmaxona – o'lchamsiz sifat (vektoral)
- .svg → Veb, tahrirlash – vektoral, kichik hajm
- .jpg → Foto – shaffoflik yo'q, siqilgan

DPI qoidasi:

dpi=72 → Ekran uchun (veb)
dpi=150 → Normal sifat
dpi=300 → Chop etish uchun (standart)
dpi=600 → Ilmiy jurnal uchun

8. Matplotlib Stillar

Mavjud stillarni ko'rish:

```
print(plt.style.available)
```

Eng mashhur stillar:

```
plt.style.use("seaborn-v0_8")      # professional, aniq
plt.style.use("ggplot")           # R tilidan ilhomlangan
plt.style.use("fivethirtyeight")  # zamonaviy, ko'zga yoqimli
plt.style.use("bmh")              # ilmiy
plt.style.use("dark_background")  # qorong' fon
```

Vaqtinchalik stil (faqat shu blok uchun):

```
with plt.style.context("seaborn-v0_8"):
    fig, ax = plt.subplots()
    ax.plot(x, y)
    plt.show()
```

9. PyQt5 ga Joylashtirish

Matplotlib grafikini PyQt5 oynasiga joylashtirishning to'g'ri usuli:

```
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg
from matplotlib.figure import Figure
```

```
class GrafikCanvas(FigureCanvasQTAgg):
    def __init__(self):
        self.fig, self.ax = plt.subplots(figsize=(8, 5))
        super().__init__(self.fig)

    def yangilashtirish(self, x, y):
        self.ax.clear()
        self.ax.plot(x, y, color="steelblue", linewidth=2)
        self.ax.set_title("Grafik")
        self.ax.grid(True, alpha=0.3)
        self.fig.tight_layout()
        self.draw()  # ← PyQt5 ga qayta chizish signali
```

QWidget ga qo'shish:

```
canvas = GrafikCanvas()
tartib.addWidget(canvas)  # oddiy widget kabi!
```

10. Xulosa

Grafik turlari:

plot() → Vaqt bo'yicha o'zgarish, tendensiya

hist() → Ma'lumot taqsimoti, chastota
scatter() → Ikki o'zgaruvchi korrelyatsiyasi

Elementlar:

set_title() → Sarlavha
set_xlabel() → X o'q nomi
set_ylabel() → Y o'q nomi
legend() → Izoh
grid() → Setka

Saqlash:

savefig("grafik.png", dpi=300, bbox_inches="tight")

Asosiy qoidalar:

- ✓ fig, ax = plt.subplots() → professional usul
- ✓ ax.clear() → qayta chizishdan oldin tozalash
- ✓ tight_layout() → elementlar bir-biriga tegmasin
- ✓ bbox_inches="tight" → saqlashda chetlar to'g'ri

💡 **Eslab qol:** Grafik — faqat chiziq emas, **hikoya**. Sarlavha, o'q nomlari va legend bo'lmasa — grafik ma'nosiz. Har doim "Bu grafikni ko'rgan odam nima tushunadi?" deb o'yla!